DEVOPS ENGINEERING MASTERY

DOCKER & CONTAINER ENGINEERING

Container Fundamentals, Docker Architecture, Image Engineering & Production Container Operations

VOLUME 06 • FIRST EDITION

```
nahid@technova:~$ docker build -t technova/app:v1.4.0 .
✓ 12 layers  •  ✓ multi-stage  •  ✓ non-root user
nahid@technova:~$ docker compose -f compose.yml -f compose.prod.yml up -d
✓ api healthy  •  ✓ mongo healthy  •  ✓ web healthy
nahid@technova:~$ docker stats --no-stream
```

Docker • containerd • Linux Namespaces • Compose • MERN

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

Companion Workbook • 24 Labs • 3 Projects • Capstone

Docker Lab Workbook

DEVOPS ENGINEERING MASTERY — Volume 06 (Companion Workbook). First Edition, 9 November 2026. Docker Engine and Docker Compose v2 with a MERN example application. This workbook collects all 24 guided labs, 15 assignments, the 3 major projects and the capstone, so you can work through them beside your Docker host without carrying the whole book. Do the labs in order; each lab states its prerequisites. Never run a lab step on a production host. The instructor-style solution guide for the capstone is at the end — attempt the capstone before reading it.

ল্যাব ওয়ার্কবুক

ল্যাব ওয়ার্কবুক — Docker & Container Engineering

এই ওয়ার্কবুকটি **DevOps Engineering Mastery — Volume 06: Docker & Container Engineering** বইয়ের হাতে-কলমে অংশ। এখানে প্রতিটি ল্যাব একটি বাস্তব container কাজের ছোট সংস্করণ। পড়ে শুধু জানা যাবে না — image বানাতে হবে, container চালাতে হবে, network ভাঙতে হবে, volume হারিয়ে আবার ফিরিয়ে আনতে হবে, আর প্রোডাকশনের মতো একটি incident-ও সামলাতে হবে।

পরিবেশ (Environment): Ubuntu 24.04 LTS (VM বা নিজের ল্যাপটপে Docker Desktop), Docker Engine 26+ / Docker Desktop, Docker Compose v2 (`docker compose` কমান্ড), Git, Node.js 20 LTS, এবং TechNova MERN নমুনা অ্যাপ — `server/` (Node.js + Express + MongoDB), `client/` (React / Next.js), `nginx/` (reverse proxy)। সব ল্যাব একটি disposable VM বা আলাদা টেস্ট মেশিনে করবেন — কোনো লাইভ প্রোডাকশন সার্ভারে নয়।

> **সতর্কতা — ল্যাব বনাম প্রোডাকশন:** সব উদাহরণে কাল্পনিক মান ব্যবহার করুন — ডোমেইন `technova.example`, IP `203.0.113.x`, secret `CHANGE-ME-SECRET`। আসল পাসওয়ার্ড, API key বা token কখনো Dockerfile, image layer, Compose ফাইল বা Git-এ রাখবেন না। Secret কখনো image-এ `ENV` হিসেবে bake করবেন না। অচেনা image কখনো `--privileged` দিয়ে চালাবেন না, Docker socket (`/var/run/docker.sock`) অপ্রয়োজনে mount করবেন না, আর `docker volume rm` বা `docker system prune -a --volumes` চালানোর আগে দুবার ভাবুন — volume মুছলে ডেটাবেসের ডেটা চিরতরে যাবে।

> **কীভাবে এই ওয়ার্কবুক ব্যবহার করবেন:** ল্যাবগুলো ক্রমানুসারে করুন — প্রতিটি ল্যাব আগের ল্যাবের উপর দাঁড়িয়ে আছে। Command কপি করার আগে প্রতিটি flag-এর ব্যাখ্যা পড়ুন। প্রতিটি ল্যাবের শেষে Verification ধাপ দিয়ে ফল যাচাই করুন, তারপর Reflection প্রশ্নের উত্তর নিজে লিখুন। Cleanup ধাপটি এড়িয়ে যাবেন না — অপ্রয়োজনীয় container, image, volume ও network মুছে ফেলা container অপারেশনেরই অংশ।

সাধারণ নিয়ম

বিষয়	নিয়ম
ডোমেইন ও IP	কাল্পনিক ডোমেইন `technova.example`, `app.technova.example`; কাল্পনিক IP `203.0.113.10`। আসল ডোমেইন বা সার্ভার IP ব্যবহার করবেন না।

Secret	উদাহরণে `CHANGE-ME-SECRET` / `CHANGE-ME-MONGO-PASSWORD` লিখুন। আসল secret `.env` ফাইলে রাখুন, সেটি `.gitignore`-এ রাখুন ও `.dockerignore`-এও রাখুন।
আউটপুট	সব নমুনা টার্মিনাল আউটপুট চিত্রণমূলক (illustrative) — লেখা থাকবে “নমুনা আউটপুট — চিত্রণমূলক”। আপনার সিস্টেমে container ID, সময় ও সংস্করণ আলাদা হতে পারে, সেটাই স্বাভাবিক।
Compose	আধুনিক Docker Compose v2 ব্যবহার করুন: `docker compose up`। পুরনো `version:` key বা `docker-compose` (hyphen) কমান্ড ব্যবহার করবেন না।
নমুনা অ্যাপ	TechNova MERN অ্যাপ — `server/` (Express + MongoDB), `client/` (React/Vite), `mongo` (MongoDB), `nginx` (reverse proxy)। না থাকলে ল্যাব ৮-এর ন্যূনতম অ্যাপ দিয়ে শুরু করুন।
নিরাপত্তা	প্রোডাকশন container root user-এ চালাবেন না; non-root user, read-only filesystem যেখানে সম্ভব, resource limit ও health check ব্যবহার করুন।

প্রতিটি ল্যাবে ব্যবহৃত ইংরেজি টেকনিক্যাল শব্দ প্রথমবার বাংলায় ব্যাখ্যা করা হয়েছে। যেমন — **container** হলো একটি image থেকে চলমান, বিচ্ছিন্ন (isolated) প্রসেসের পরিবেশ; **image** হলো container চালানোর নীলনকশা — read-only layer-এর স্তূপ; **Dockerfile** হলো image বানানোর নির্দেশপত্র; **volume** হলো container-এর বাইরে স্থায়ী ডেটা রাখার জায়গা; **network** হলো container-দের মধ্যে যোগাযোগের ভার্চুয়াল নেটওয়ার্ক; **Compose** হলো একাধিক container একসাথে সংজ্ঞায়িত ও চালানোর টুল।

ল্যাবগুলোর মানচিত্র

ল্যাব ১–৬: container-এর ভিত্তি — VM বনাম container, Docker architecture, ইনস্টলেশন, প্রথম container, lifecycle ও image। ল্যাব ৭–১২: Dockerfile, Node.js অ্যাপ containerize করা, build cache, multi-stage build, image optimization ও Docker Hub-এ publish। ল্যাব ১৩–১৭: network, DNS ও service discovery, volume ও bind mount, ডেটাবেস persistence ও recovery। ল্যাব ১৮–২০: Docker Compose-এ পূর্ণ MERN পরিবেশ, environment variable, health check ও dependency। ল্যাব ২১–২৪: non-root নিরাপত্তা, resource limit ও log, update ও rollback, এবং প্রোডাকশন incident troubleshooting — অর্থাৎ সম্পূর্ণ container জীবনচক্র।

> **মূল ধারণা — কেন container?** একই অ্যাপ ডেভেলপারের ল্যাপটপে চলে, সার্ভারে চলে না — এই “আমার মেশিনে তো চলছিল” সমস্যার উত্তর container। Image-এ অ্যাপ, তার dependency, runtime ও configuration একসাথে বাঁধা থাকে; যেখানে Docker আছে, সেখানেই একইভাবে চলে। VM পুরো operating system নিয়ে চলে, container host-এর kernel ভাগ করে নেয় — তাই হালকা, দ্রুত শুরু হয়, আর এক মেশিনে অনেক container চলে।

ল্যাব 1

VM ও Container আচরণ তুলনা করা

উদ্দেশ্য (Objective): একই কাজ — একটি ছোট ওয়েব সার্ভার চালানো — virtual machine (VM) ও container-এ করে শুরু হওয়ার সময়, মেমরি ব্যবহার, disk ব্যবহার ও বিচ্ছিন্নতা (isolation) নিজে মেপে দেখা। এই ল্যাবের পর আপনি সংখ্যা দিয়ে বলতে পারবেন কেন container হালকা ও দ্রুত, আর কখন VM-ই সঠিক পছন্দ।

পূর্বশর্ত (Prerequisites): ল্যাবের আগের কোনো পূর্বশর্ত নেই; টার্মিনাল চালাতে পারেন; Ubuntu 24.04 LTS-এ Docker Engine ইনস্টল আছে (না থাকলে ল্যাব ৩ আগে করুন, তবে তুলনার ধারণা বুঝতে এখন পড়ে নিন)।

প্রয়োজনীয় পরিবেশ (Required Environment): Ubuntu 24.04 LTS VM বা হোস্ট (কমপক্ষে 4 GB RAM, 20 GB ফাঁকা disk), Docker Engine 26+, `time` কমান্ড, `free -h`, `df -h`। তুলনার জন্য একটি ছোট Linux VM (VirtualBox/UTM) থাকলে ভালো, না থাকলে হোস্টের সংখ্যা ও container-এর সংখ্যা তুলনা করুন।

আর্কিটেকচার:

VM: Hardware → Hypervisor → Guest OS (পূর্ণ kernel + library) → App | Container: Hardware → Host OS + Kernel → Docker Engine → Container (শুধু অ্যাপ + library, kernel ভাগ করে নেয়)

VM boot: মিনিট | Container start: সেকেন্ডের ভগ্নাংশ

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# --- VM বনাম container: মাপার কমান্ড ---
# হোস্টের মেমরি ও disk দেখুন
free -h && df -h /

# একটি container চালিয়ে শুরু হওয়ার সময় মাপুন
time docker run --rm nginx:1.27-alpine echo "container started"
# নমুনা আউটপুট — চিত্রণমূলক
# real  0m1.234s
# container started

# চলমান container-এর মেমরি ব্যবহার দেখুন
docker run -d --name web-compare nginx:1.27-alpine
docker stats --no-stream web-compare
# নমুনা আউটপুট — চিত্রণমূলক
# CONTAINER ID   NAME         CPU %   MEM USAGE / LIMIT
# a1b2c3d4e5f6   web-compare  0.00%   2.1MiB / 3.8GiB

# image-এর আকার দেখুন — VM disk image-এর সঙ্গে তুলনা করুন
docker images nginx:1.27-alpine
# নমুনা আউটপুট — চিত্রণমূলক
# REPOSITORY   TAG           IMAGE ID       SIZE
# nginx        1.27-alpine   ab12cd34ef56   47.3MB
# একটি Ubuntu VM disk image সাধারণত 2–10 GB হয় — প্রায় 50–200 গুণ বড়।
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: হোস্টে `free -h` ও `df -h` চালিয়ে বেসলাইন লিখে রাখুন — কত RAM ফাঁকা, কত disk ফাঁকা।

ধাপ 2: `time docker run --rm nginx:1.27-alpine echo ...` চালিয়ে container শুরুর সময় মাপুন — তিনবার চালিয়ে গড় নিন।

ধাপ 3: VM থাকলে সেটি boot করতে কত সময় লাগে তা ঘড়ি দিয়ে মাপুন (power on থেকে login prompt পর্যন্ত); না থাকলে নোট করুন যে VM boot সাধারণত 30–120 সেকেন্ড লাগে।

ধাপ 4: `docker run -d --name web-compare nginx:1.27-alpine` চালিয়ে `docker stats --no-stream` দিয়ে মেমরি লিখুন।

ধাপ 5: `docker images` দিয়ে nginx image-এর আকার লিখুন, আর VM disk ফাইলের আকারের সঙ্গে তুলনা করুন।

ধাপ 6: একই মেশিনে 5টি nginx container চালান (`docker run -d nginx:1.27-alpine` পাঁচবার) — মোট মেমরি দেখুন; 5টি VM চালানো কল্পনা করলে কী হতো তা লিখুন।

ধাপ 7: বিচ্ছিন্নতা পরীক্ষা করুন: container-এর ভেতরে `docker exec web-compare ps aux` চালান — হোস্টের প্রসেস দেখা যায় কি না দেখুন।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker ps
docker stats --no-stream
docker images nginx:1.27-alpine
# প্রত্যাশিত: container 1–3 সেকেন্ডে শুরু হয়, মেমরি কয়েক MB,
# image ~47 MB; VM-এর তুলনায় দুই মাত্রাতেই অনেক ছোট।
```

প্রত্যাশিত ফলাফল: Container ১–৩ সেকেন্ডে শুরু হয়, মেমরি কয়েক MB খায়, image মাত্র কয়েক দশ MB; VM boot-এ মিনিট লাগে, GB-খানেক RAM ও disk খায়। তবে container হোস্টের প্রসেস দেখতে পায় না — বিচ্ছিন্নতা আছে, কিন্তু VM-এর মতো পূর্ণ kernel বিচ্ছিন্নতা নয়। আপনার নোটে সময়, মেমরি ও disk-এর তিনটি সংখ্যাসহ তুলনা টেবিল থাকবে।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
VM-এর মতো container-এ আলাদা kernel আশা করা	Container host kernel ভাগ করে নেয় — `uname -r` হোস্টের kernel-ই দেখাবে; এটাই স্বাভাবিক, ভুল নয়।
প্রথম `docker run` ধীর লাগা	প্রথমবার image download হয়; দ্বিতীয়বার cache থেকে চলে — `docker images` দিয়ে দেখুন image নেমেছে কি না।
`docker stats` -এ MEM LIMIT হোস্টের RAM দেখানো	এটি হোস্টের মোট RAM, container-এর limit নয় — limit না দিলে container হোস্টের সব RAM ব্যবহার করতে পারে; ল্যাব ২২-এ limit দেওয়া শিখবেন।

Troubleshooting

ফলাফল না মিললে প্রথমে `docker info` দিয়ে Docker চলছে কি না দেখুন, তারপর `docker ps -a` দিয়ে container-এর অবস্থা দেখুন। সংখ্যা অদ্ভুত লাগলে পরিমাপটি তিনবার পুনরাবৃত্তি করুন — প্রথম run-এ image pull-এর সময় মিশে থাকে, তাই গড় নেওয়াই সঠিক পদ্ধতি। VM না থাকলে তুলনাটি নথিভুক্ত প্রকাশিত সংখ্যা (VM boot 30–120 সেকেন্ড, disk কয়েক GB) দিয়ে করুন এবং সেটি অনুমান বলে স্পষ্ট লিখুন।

Cleanup

`docker stop web-compare && docker rm web-compare` এবং অতিরিক্ত পাঁচটি container `docker rm -f $(docker ps -aq)` দিয়ে মুছুন — তবে এই কমান্ড শুধু এই ল্যাবের container থাকলে চালাবেন; অন্য ল্যাবের container থাকলে নাম ধরে মুছুন।

Reflection প্রশ্ন

১. একই মেশিনে 50টি container চালানো সম্ভব হলেও 50টি VM কেন সম্ভব নয়? ২. Container হোস্ট kernel ভাগ করে নেয় — এর নিরাপত্তা-ঝুঁকি কী, আর কখন VM বেছে নেবেন? ৩. শুরুর সময় ছাড়াও আর কোন মাপে (যেমন density, portability) container জেতে?

ল্যাব 2

Docker Architecture অন্বেষণ করা

উদ্দেশ্য (Objective): Docker-এর তিনটি মূল অংশ — client, daemon (dockerd) ও registry — এবং image, container, volume, network-এর সম্পর্ক হাতে-কলমে দেখা। এই ল্যাবের পর `docker run` দিলে ভেতরে কী কী ঘটে, তা ধাপে ধাপে বলতে পারবেন।

পূর্বশর্ত (Prerequisites): ল্যাব ১ সম্পূর্ণ; Docker ইনস্টল আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Ubuntu 24.04 LTS + Docker Engine, টার্মিনাল, ইন্টারনেট সংযোগ (image pull-এর জন্য)।

আর্কিটেকচার:
User → Docker Client (CLI: docker) → REST API / Unix socket → Docker Daemon (dockerd) → Containerd → runc → Container
Daemon → Registry (Docker Hub): image pull / push | Daemon → Images / Volumes / Networks (হোস্টে সংরক্ষিত)

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# Docker daemon ও client-এর সংস্করণ দেখুন
docker version
# নমুনা আউটপুট — চিত্রণমূলক
# Client: Docker Engine - Community  Version: 26.1.0
# Server: Docker Engine - Community  Version: 26.1.0

# daemon-এর তথ্য: কত container, image, storage driver
docker info

# daemon কোথায় শুনছে (socket) দেখুন
ls -l /var/run/docker.sock
sudo systemctl status docker --no-pager | head -20

# image কোথা থেকে আসে: registry-র ইতিহাস দেখুন
docker pull hello-world
docker image inspect hello-world --format '{{.RepoDigests}}'
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `docker version` চালিয়ে client ও server সংস্করণ আলাদা করে দেখুন — দুটো আলাদা প্রোগ্রাম, এটাই client-server architecture।

ধাপ 2: `docker info` চালিয়ে `Storage Driver` (সাধারণত `overlay2`), `Logging Driver` ও চলমান container/image-এর সংখ্যা লিখুন।

ধাপ 3: `ls -l /var/run/docker.sock` দিয়ে দেখুন socket-টি কার মালিকানায় ও কী permission-এ আছে — এটিই Docker API-এর দরজা, তাই এর নিরাপত্তা গুরুত্বপূর্ণ।

ধাপ 4: `sudo systemctl status docker` দিয়ে daemon চলছে কি না, কোন user-এ চলছে দেখুন।

ধাপ 5: `docker pull hello-world` চালান ও লক্ষ্য করুন daemon image-টি Docker Hub (registry) থেকে নামায়, তারপর `docker run hello-world` চালান।

ধাপ 6: `docker image inspect hello-world` দিয়ে image-এর layer, architecture ও digest দেখুন — digest-ই image-এর অপরিবর্তনীয় পরিচয়।

ধাপ 7: একটি ছোট চিত্র আঁকুন: আপনার `docker run` কমান্ডটি client থেকে daemon, daemon থেকে containerd/runc, আর registry পর্যন্ত কীভাবে যায়।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker version
docker info | grep -E 'Server Version|Storage Driver|Containers|Images'
docker image inspect hello-world --format 'Digest: {{index .RepoDigests 0}}'
# প্রত্যাশিত: client ও server দুটোর সংস্করণ দেখা যায়; storage driver overlay2;
# hello-world চলে ও "Hello from Docker!" ছাপে।
```

প্রত্যাশিত ফলাফল: Client ও server আলাদা সংস্করণ দেখায়, `docker info`-তে overlay2 storage driver ও container/image-এর গণনা মেলে, hello-world container চলে। আপনি ব্যাখ্যা করতে পারবেন: CLI শুধু নির্দেশ দেয়, আসল কাজ daemon করে, image আসে registry থেকে।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
Client সংস্করণ দেখা গেল, server দেখা গেল না	Daemon চলছে না — `sudo systemctl start docker` দিন, তারপর `docker version` আবার চালান।
`permission denied` socket-এ	আপনার user `docker` group-এ নেই — ল্যাব ৩-এ group যোগ করা আছে; আপাতত `sudo docker ...` ব্যবহার করুন।
`docker info`-তে storage driver ভিন্ন	overlay2-ই আধুনিক ডিফল্ট; অন্য driver দেখালে সেটি আপনার filesystem-এর কারণে হতে পারে — ল্যাবে সমস্যা নয়, নোট করে রাখুন।

Troubleshooting

Daemon সাড়া না দিলে `sudo journalctl -u docker --no-pager | tail -30` দিয়ে daemon-এর log দেখুন — বেশিরভাগ সমস্যার কারণ সেখানেই লেখা থাকে। `docker context ls` দিয়ে দেখুন আপনি কোন daemon-এর সঙ্গে কথা বলছেন (local না remote) — ভুল context-এ কমান্ড গেলে ফল অদ্ভুত লাগবে।

Cleanup

`docker rm $(docker ps -aq) 2>/dev/null; docker rmi hello-world` — hello-world image মুছে ফেলুন; daemon বন্ধ করবেন না, পরের ল্যাবগুলোতে লাগবে।

Reflection প্রশ্ন

১. CLI আর daemon আলাদা থাকার সুবিধা কী — remote Docker host কীভাবে কাজ করে? ২. Docker socket-এ যার access আছে, সে কার্যত কী করতে পারে — কেন socket শেয়ার করা বিপজ্জনক? ৩. Registry ছাড়া কি Docker চলবে? image কোথা থেকে আসত তখন?

ল্যাব 3

Docker Engine ইনস্টল ও যাচাই করা

উদ্দেশ্য (Objective): Ubuntu 24.04 LTS-এ অফিসিয়াল Docker repository থেকে Docker Engine, CLI, containerd ও Compose plugin ইনস্টল করা, non-root user-এ চালানো ও সম্পূর্ণ যাচাই করা। এই ল্যাবের পর যেকোনো নতুন Ubuntu মেশিনে আপনি নিজে Docker বসাতে পারবেন।

পূর্বশর্ত (Prerequisites): Ubuntu 24.04 LTS VM বা হোস্টে sudo access আছে; ইন্টারনেট সংযোগ আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Ubuntu 24.04 LTS (ফাঁকা VM-ই ভালো), sudo user, ইন্টারনেট, কমপক্ষে 2 GB RAM।

আর্কিটেকচার:
APT → Docker অফিসিয়াল repository → apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin → dockerd (systemd) → docker group-এ user → docker run hello-world (যাচাই)

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# পুরনো, সংঘাতময় প্যাকেজ সরান (না থাকলে কিছু হবে না)
sudo apt remove -y docker.io docker-doc docker-compose docker-compose-v2 podman-docker
containerd runc || true
sudo apt update && sudo apt install -y ca-certificates curl

# Docker-এর GPG key ও অফিসিয়াল repository যোগ করুন
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://
download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" |
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# ইনস্টল করুন
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-
plugin

# নিজের user-কে docker group-এ যোগ করুন (sudo ছাড়া docker চালাতে)
sudo usermod -aG docker "$USER"
# group কার্যকর করতে logout/login করুন, অথবা আপাতত:
newgrp docker

# যাচাই
docker --version
docker compose version
docker run hello-world
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: VM-এর snapshot নিন (থাকলে) — ইনস্টলেশন ব্যর্থ হলে ফিরে আসা যায়।

ধাপ 2: পুরনো Docker প্যাকেজ সরানোর কমান্ড চালান, তারপর `ca-certificates` ও `curl` ইনস্টল করুন।

ধাপ 3: GPG key ও repository যোগ করুন — key ছাড়া repository যোগ করবেন না, কারণ তাতে যাচাই না করা প্যাকেজ ইনস্টল হওয়ার ঝুঁকি থাকে।

ধাপ 4: `sudo apt update`-এর পর পাঁচটি প্যাকেজ একসাথে ইনস্টল করুন; কোনো প্যাকেজ missing বললে repository লাইনটি ঠিক আছে কি না দেখুন।

ধাপ 5: `sudo usermod -aG docker $USER` চালিয়ে logout/login করুন (বা `newgrp docker`), তারপর sudo ছাড়া `docker ps` চালিয়ে দেখুন।

ধাপ 6: `docker --version`, `docker compose version` ও `docker run hello-world` — তিনটি যাচাই লিখে রাখুন।

ধাপ 7: `sudo systemctl enable --now docker` দিয়ে daemon boot-এ চালু হওয়া নিশ্চিত করুন।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker --version
# নমুনা আউটপুট — চিত্রণমূলক: Docker version 26.1.0, build 1234567
docker compose version
# নমুনা আউটপুট — চিত্রণমূলক: Docker Compose version v2.27.0
docker run hello-world
# নমুনা আউটপুট — চিত্রণমূলক: Hello from Docker!
```

প্রত্যাশিত ফলাফল: `docker --version`, `docker compose version` ও hello-world তিনটিই সফল; sudo ছাড়া `docker ps` চলে; daemon reboot-এর পরেও চালু থাকার জন্য enable করা আছে।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
GPG key ডাউনলোড ব্যর্থ	ইন্টারনেট/প্রক্সি দেখুন; `curl -fsSL`-এর URL টাইপ ঠিক আছে কি না মিলিয়ে দেখুন।
প্যাকেজ `docker-ce` পাওয়া যাচ্ছে না	`cat /etc/apt/sources.list.d/docker.list` দিয়ে codename (`noble`) ঠিক আছে কি না দেখুন — ভুল Ubuntu সংস্করণের repository যোগ হলে এমন হয়।
Logout-এর পরেও `permission denied`	Group সদস্যতা নতুন login-এ কার্যকর হয় — সম্পূর্ণ logout/login করুন, শুধু টার্মিনাল বন্ধ করলেই হবে না।
hello-world-এ `Unable to find image` তারপর pull ব্যর্থ	Registry-তে পৌঁছানো যাচ্ছে না — DNS/প্রক্সি দেখুন, `curl https://hub.docker.com` পরীক্ষা করুন।

Troubleshooting

ইনস্টলেশন আটকে গেলে `sudo apt update`-এর আউটপুটের শেষ লাইনগুলো পড়ুন — repository-র ভুল প্রায় সবসময় সেখানেই ধরা পড়ে। Docker Desktop ব্যবহার করলে Engine ইনস্টলের এই ধাপগুলো প্রযোজ্য নয়; সেক্ষেত্রে Docker Desktop-এর Settings → Resources যাচাই করে `docker version` দিয়ে এগোন।

Cleanup

hello-world container নিজে থেকেই থেমে যায়, মুছতে `docker rm $(docker ps -aq)` চালান; image রাখুন, পরের ল্যাবে লাগতে পারে। ইনস্টল করা Docker রাখুন — পুরো ওয়ার্কবুকের ভিত্তি এটাই।

Reflection প্রশ্ন

১. Ubuntu-র নিজের `docker.io` প্যাকেজের বদলে অফিসিয়াল repository কেন ভালো? ২. `docker` group-এ থাকা মানে কার্যত root-এর সমান ক্ষমতা — কেন? ৩. `docker-compose-plugin` কী দেয়, যা Engine একা দেয় না?

ল্যাব 4

প্রথম Container চালানো

উদ্দেশ্য (Objective): `docker run`-এর মূল flag-গুলো — `-d`, `--name`, `-p`, `-it`, `--rm` — ব্যবহার করে nginx ওয়েব সার্ভার container চালিয়ে ব্রাউজার থেকে দেখা। এই ল্যাবের পর যেকোনো image থেকে container চালিয়ে সেটি বাইরে থেকে access করতে পারবেন।

পূর্বশর্ত (Prerequisites): ল্যাব ৩ সম্পূর্ণ; Docker চলছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, ইন্টারনেট (nginx image pull-এর জন্য), হোস্টে ফাঁকা port 8080, ব্রাউজার বা `curl`।

আর্কিটেকচার:
Host (curl/ব্রাউজার :8080) → Port mapping 8080:80 → Container nginx:1.27-alpine (port 80) → 'Welcome to nginx!' পাতা

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# nginx container detached mode-এ, নাম দিয়ে, port map করে চালান
docker run -d --name technova-web -p 8080:80 nginx:1.27-alpine

# চলছে কি না দেখুন
docker ps
curl -s http://localhost:8080 | head -5
# নমুনা আউটপুট — চিত্রণমূলক
#

# interactive container: ভেতরে ঢুকে দেখুন
docker run -it --rm alpine:3.20 sh
# ভেতরে: hostname; cat /etc/os-release; exit
# নমুনা আউটপুট — চিত্রণমূলক: container-এর hostname একটি এলোমেলো ID

# container-এর log দেখুন
docker logs technova-web
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `docker run -d --name technova-web -p 8080:80 nginx:1.27-alpine` চালান — `-d` মানে detached (পেছনে চলবে), `-p 8080:80` মানে হোস্টের 8080 → container-এর 80।

ধাপ 2: `docker ps` দিয়ে STATUS `Up` দেখুন, তারপর `curl http://localhost:8080` বা ব্রাউজারে গিয়ে nginx-এর স্বাগত পাতা দেখুন।

ধাপ 3: `docker run -it --rm alpine:3.20 sh` চালিয়ে container-এর ভেতরে shell নিন — `hostname`, `cat /etc/os-release` চালান, তারপর `exit` করুন; `--rm`-এর কারণে container নিজে মুছে যাবে।

ধাপ 4: চলমান nginx-এ `docker exec -it technova-web sh` দিয়ে ঢুকুন, `ls /usr/share/nginx/html` দেখুন, `exit` করুন — exec মানে চলমান container-এ নতুন প্রসেস চালানো।

ধাপ 5: `docker logs technova-web` দিয়ে access log দেখুন — আপনার curl request-এর দাগ আছে কি না মিলিয়ে দেখুন।

ধাপ 6: `docker stop technova-web` দিয়ে থামান, `curl` আবার চালিয়ে দেখুন সংযোগ ব্যর্থ হয়, তারপর `docker start technova-web` দিয়ে আবার চালান।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080
# প্রত্যাশিত: STATUS Up, Ports 0.0.0.0:8080->80/tcp, HTTP 200
# নমুনা আউটপুট — চিত্রণমূলক: 200
```

প্রত্যাশিত ফলাফল: `docker ps`-এ `technova-web` Up, port 8080→80 mapped, `curl`-এ HTTP 200 ও nginx পাতা আসে; interactive container `--rm`-এর কারণে exit-এর পর নিজে মুছে যায়; stop/start-এ container আবার চলে।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
Port already in use (8080)	হোস্টে 8080 ব্যস্ত — `ss -tlnp | grep 8080` দেখুন, বা অন্য port ব্যবহার করুন: `-p 8081:80`।
curl-এ connection refused	Container Up কি না `docker ps`-এ দেখুন; `docker logs technova-web`-এ nginx error আছে কি না দেখুন।
`docker exec`-এ shell পাওয়া যাচ্ছে না	alpine image-এ `bash` নেই, `sh` ব্যবহার করুন — এটাই সঠিক shell।
Container সঙ্গে সঙ্গে Exited	`docker logs <name>` দেখুন — মূল প্রসেস থেমে গেলে container থামে; nginx-এর ক্ষেত্রে এমন হওয়ার কথা নয়।

Troubleshooting

Container না চললে প্রথমে `docker ps -a` দিয়ে দেখুন সেটি Exited কি না, তারপর `docker logs` দেখুন — log-এই কারণ লেখা থাকে। Port সমস্যা হলে `docker port technova-web` দিয়ে আসল mapping যাচাই করুন; ভুল mapping-এ `curl` হোস্টের ভুল port-এ যাবে।

Cleanup

`docker stop technova-web && docker rm technova-web`। পরের ল্যাবে lifecycle নিয়ে কাজ হবে, তাই image (`nginx:1.27-alpine`) রেখে দিন।

Reflection প্রশ্ন

১. `-d` ছাড়া `docker run nginx` চালালে টার্মিনালে কী হবে, আর কেন? ২. `docker run` বনাম `docker exec`-এর পার্থক্য কী? ৩. Port mapping না দিলে হোস্ট থেকে container-এর ওয়েব সার্ভারে পৌঁছানো যায় না কেন?

ল্যাব 5

Container Lifecycle পরিচালনা করা

উদ্দেশ্য (Objective): Container-এর পুরো জীবনচক্র — create, start, stop, restart, pause/unpause, kill, rm — হাতে-কলমে চালানো ও প্রতিটি অবস্থা (`created`, `running`, `exited`, `paused`) দেখা। এই ল্যাবের পর থেমে যাওয়া container দেখে আপনি বলতে পারবেন সেটি স্বাভাবিকভাবে থেমেছে, না crash করেছে।

পূর্বশর্ত (Prerequisites): ল্যাব ৪ সম্পূর্ণ।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, nginx ও alpine image।

আর্কিটেকচার:

docker create → docker start → running → docker pause → paused → docker unpause → running → docker stop (SIGTERM, 10s grace) → exited → docker start (আবার) / docker rm (মোছা)

docker kill = SIGKILL, সঙ্গে সঙ্গে থামে, grace period নেই

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# create করে পরে start করা (run = create + start একসাথে)
docker create --name lifecycle-demo nginx:1.27-alpine
docker ps -a --filter name=lifecycle-demo
docker start lifecycle-demo

# pause / unpause
docker pause lifecycle-demo
docker ps --filter name=lifecycle-demo
docker unpause lifecycle-demo

# restart ও stop-এর সময় মাপা
time docker restart lifecycle-demo
time docker stop lifecycle-demo
docker ps -a --filter name=lifecycle-demo

# exit code দেখা — স্বাভাবিক stop-এ 0, kill-এ 137
docker start lifecycle-demo
docker kill lifecycle-demo
docker inspect lifecycle-demo --format 'Exit: {{.State.ExitCode}} Status: {{.State.Status}}'
# নমুনা আউটপুট — চিত্রণমূলক: Exit: 137 Status: exited
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `docker create --name lifecycle-demo nginx:1.27-alpine` চালিয়ে `docker ps -a`-তে `Created` অবস্থা দেখুন — create করলেই container চলে না।

ধাপ 2: `docker start` দিয়ে চালান, `docker ps`-তে `Up` দেখুন।

ধাপ 3: `docker pause` দিয়ে থামান (প্রসেস freeze হয়, মরে না), `docker ps`-তে `Paused` দেখুন, তারপর `unpause` করুন।

ধাপ 4: `time docker restart` ও `time docker stop` চালিয়ে দেখুন stop-এ কয়েক সেকেন্ড লাগে (graceful SIGTERM), restart-এ stop+start দুটোই হয়।

ধাপ 5: `docker kill` চালিয়ে `docker inspect --format '{{.State.ExitCode}}'` দেখুন — 137 মানে SIGKILL-এ মারা গেছে।

ধাপ 6: একটি container-কে স্বাভাবিকভাবে থামান (alpine-এ `exit 0`) আরেকটিকে kill করুন — দুটোর exit code পাশাপাশি লিখুন: 0 বনাম 137 বনাম 1।

ধাপ 7: `docker rm lifecycle-demo` দিয়ে মুছুন; চলমান container মুছতে `-f` লাগে কেন, তা পরীক্ষা করে দেখুন।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker ps -a --filter name=lifecycle-demo --format 'table {{.Names}}\t{{.Status}}'
docker inspect lifecycle-demo --format 'ExitCode={{.State.ExitCode}}'
# প্রত্যাশিত: stop-এর পর Status Exited (0), kill-এর পর ExitCode 137
```

প্রত্যাশিত ফলাফল: প্রতিটি অবস্থা — Created, Up, Paused, Exited (0), Exited (137) — `docker ps -a` ও `docker inspect`-এ দেখা যায়; `stop` graceful, `kill` তাৎক্ষণিক; `rm` শুধু থেমে যাওয়া container মোছে।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
চলমান container `docker rm`-এ মোছা যাচ্ছে না	স্বাভাবিক — প্রথমে `docker stop` দিন, বা `docker rm -f` ব্যবহার করুন (তবে -f মানে graceful shutdown বাদ)।
`docker stop` 10 সেকেন্ডের বেশি লাগছে	অ্যাপ SIGTERM ধরছে না — 10 সেকেন্ড পর Docker SIGKILL দেয়; ল্যাব ২০-এর graceful shutdown আলোচনা দেখুন।
Pause করা container-এ `exec` ব্যর্থ	Paused container-এ নতুন প্রসেস চলে না — আগে `unpause` করুন।
Exit code 0 অথচ অ্যাপ চলছে না	মূল প্রসেস শেষ হলেই container থামে — `docker run`-এ background-এ কিছু চালিয়ে foreground খালি রাখলে container সঙ্গে সঙ্গে থামে।

Troubleshooting

Container হঠাৎ থেমে গেলে `docker inspect <name>`-এর `State` অংশে `ExitCode`, `OOMKilled` ও `Error` দেখুন — OOMKilled true মানে মেমরি শেষ হয়ে গেছে, সেটি ল্যাব ২২-এর বিষয়। Log না থাকলে `docker logs --tail 50` দেখুন; container মুছে ফেললে log-ও যায়, তাই তদন্তের আগে মুছবেন না।

Cleanup

`docker rm lifecycle-demo` (থেমে থাকলে) — নামটি পরের ল্যাবে আবার ব্যবহার হবে না, তাই মুছে ফেলা নিরাপদ।

Reflection প্রশ্ন

১. `docker stop`-এর 10 সেকেন্ড grace period কেন দরকার — ডেটাবেস container-এ এর গুরুত্ব কী? ২. Exit code 137, 1 ও 0 — প্রতিটি কী বলছে? ৩. `restart: always` কখন সাহায্য করে, আর কখন crash-loop ঢাকে?

ল্যাব 6

Docker Image পরিদর্শন করা

উদ্দেশ্য (Objective): Image-এর layer, tag, digest, আকার ও ইতিহাস পরীক্ষা করা — `docker images`, `inspect`, `history` ও `tag` ব্যবহার করে। এই ল্যাবের পর আপনি বুঝবেন `latest` tag কেন ভরসাযোগ্য নয়, আর layer কীভাবে cache ও আকার নির্ধারণ করে।

পূর্বশর্ত (Prerequisites): ল্যাব ৪–৫ সম্পূর্ণ; nginx ও alpine image ডাউনলোড করা আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, nginx:1.27-alpine, alpine:3.20, ইন্টারনেট (ঐচ্ছিক, tag পরীক্ষার জন্য)।

আর্কিটেকচার:
Registry → Image (read-only layer-এর স্তূপ: base OS → package → config → app) → Tag (মানুষের পড়ার নাম, যেমন 1.27-alpine) → Digest (sha256:..., অপরিবর্তনীয় পরিচয়) → Container (image + লেখার মতো thin layer)

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# সব image তালিকা: আকার, tag, তৈরির সময়
docker images
docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'

# একটি image-এর layer ইতিহাস দেখুন
docker history nginx:1.27-alpine
# নমুনা আউটপুট — চিত্রণমূলক: প্রতিটি Dockerfile নির্দেশ একটি layer

# image-এর বিস্তারিত তথ্য
docker inspect nginx:1.27-alpine --format 'Arch: {{.Architecture}} OS: {{.Os}} Size:
{{.Size}}'
docker inspect nginx:1.27-alpine | grep -i digest

# tag করা: একই image-কে নিজের নামে চিহ্নিত করা
docker tag nginx:1.27-alpine technova-web:1.0.0
docker images technova-web

# dangling (নামহীন) image দেখুন
docker images -f dangling=true
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `docker images` চালিয়ে প্রতিটি image-এর repository, tag, image ID ও আকার লিখুন।

ধাপ 2: `docker history nginx:1.27-alpine` দিয়ে দেখুন কয়টি layer আছে ও কোন layer সবচেয়ে বড় — সাধারণত base OS ও package layer বড় হয়।

ধাপ 3: `docker inspect` দিয়ে architecture (`amd64` না `arm64`), OS ও digest লিখুন — digest-ই প্রমাণ করে image-টি ঠিক কোনটি।

ধাপ 4: `docker tag nginx:1.27-alpine technova-web:1.0.0` চালিয়ে দেখুন নতুন tag দিলে image কপি হয় না, একই image ID-তে আরেকটি নাম যোগ হয়।

ধাপ 5: `nginx` ও `nginx:latest` একই কি না `docker images`-এ image ID মিলিয়ে দেখুন — `latest` শুধু একটি tag, কোনো সংস্করণের গ্যারান্টি নয়।

ধাপ 6: দুটি ভিন্ন tag-এর একই image থেকে দুটি container চালান — দুটোই চলে, কারণ tag শুধু নাম।

ধাপ 7: `docker images -f dangling=true` দিয়ে নামহীন layer দেখুন — এগুলো পুরনো build-এর অবশিষ্টাংশ।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker images
docker history nginx:1.27-alpine --no-trunc | head -10
docker inspect nginx:1.27-alpine --format '{{.RepoDigests}}'
# প্রত্যাশিত: layer তালিকা, digest sha256:... দিয়ে শুরু, technova-web:1.0.0-এর
# image ID nginx:1.27-alpine-এর সমান।
```

প্রত্যাশিত ফলাফল: Layer ইতিহাসে প্রতিটি Dockerfile নির্দেশ দেখা যায়, digest দিয়ে image শনাক্ত করা যায়, tag করা image-এর ID একই থাকে — প্রমাণ হয় tag শুধু একটি উপনাম। `latest`-এর বিপদটিও স্পষ্ট: আজকের `latest` আর আগামী মাসের `latest` ভিন্ন image হতে পারে।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
একই image দুবার দেখাচ্ছে, আকার দ্বিগুণ মনে হচ্ছে	না — tag শুধু নাম, disk-এ image একবারই আছে; `docker system df` দিয়ে আসল disk ব্যবহার দেখুন।
`docker history` খালি দেখাচ্ছে	Image ID ভুল টাইপ করেছেন কি না দেখুন; `docker images` থেকে ID কপি করুন।
Digest খালি আসছে	স্থানীয়ভাবে build/tag করা image-এ RepoDigest থাকে না — registry থেকে pull করা image-এ থাকে; এটাই স্বাভাবিক।
Dangling image অনেক জমেছে	`docker image prune` দিয়ে মুছুন — তবে ব্যবহৃত image মুছবে না, শুধু নামহীনগুলো যাবে।

Troubleshooting

Image-এর আকার অস্বাভাবিক বড় মনে হলে `docker history --human` দিয়ে কোন layer বড় তা খুঁজুন — ল্যাব ১১-এ সেটি ছোট করার কৌশল আছে। ভুল image চালিয়ে ফেললে `docker inspect`-এর digest ও `Created` তারিখ মিলিয়ে দেখুন — tag বদলাতে পারে, digest বদলায় না।

Cleanup

`docker rmi technova-web:1.0.0` — শুধু tag মুছুন, মূল nginx image রাখুন। Dangling image থাকলে `docker image prune -f` চালাতে পারেন।

Reflection প্রশ্ন

১. Tag বনাম digest — প্রোডাকশনে কোনটি দিয়ে image নির্দিষ্ট করা উচিত, এবং কেন? ২. Layer ভাগ করে নেওয়া (sharing) কীভাবে disk বাঁচায় — দুটি image একই base ব্যবহার করলে কী হয়? ৩. `latest` tag প্রোডাকশনে ব্যবহার করলে rollback-এ কী সমস্যা হয়?

ল্যাব 7

মৌলিক Dockerfile লেখা

উদ্দেশ্য (Objective): প্রথম Dockerfile লেখা — `FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE`, `CMD` — এবং `docker build` দিয়ে নিজের image বানানো। এই ল্যাবের পর আপনি যেকোনো স্ট্যাটিক ওয়েবসাইটকে image-এ বাঁধতে পারবেন।

পূর্বশর্ত (Prerequisites): ল্যাব ৬ সম্পূর্ণ; Dockerfile নির্দেশগুলোর নাম শুনেছেন।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, টেক্সট এডিটর, একটি ফাঁকা ফোল্ডার `~/technova-lab7/`।

আর্কিটেকচার:
Build context (Dockerfile + ফাইল) → docker build → Image layer: FROM nginx → COPY html → Image: technova-site:1.0 → docker run -p 8080:80 → ব্রাউজার

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# ~/technova-lab7/ ফোল্ডারে এই দুটি ফাইল রাখুন
# --- index.html ---
#
```

```
# --- Dockerfile ---
FROM nginx:1.27-alpine
WORKDIR /usr/share/nginx/html
COPY index.html ./
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

# build করুন (শেষের . মানে build context বর্তমান ফোল্ডার)
docker build -t technova-site:1.0 .
# নমুনা আউটপুট — চিত্রণমূলক
# [+] Building 2.1s (7/7) FINISHED
#  => [1/3] FROM docker.io/library/nginx:1.27-alpine
#  => [2/3] COPY index.html ./
#  => [3/3] ... exporting to image

docker run -d --name my-site -p 8080:80 technova-site:1.0
curl -s http://localhost:8080
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `mkdir -p ~/technova-lab7 && cd ~/technova-lab7` তৈরি করুন, `index.html`-এ TechNova-র স্বাগত লেখা লিখুন।

ধাপ 2: উপরের Dockerfile লিখুন — প্রতিটি লাইনের পাশে নিজের ভাষায় মন্তব্য লিখুন: `FROM` base image নেয়, `COPY` ফাইল image-এ তোলে, `CMD` container চালু হলে কী চলবে তা বলে।

ধাপ 3: `docker build -t technova-site:1.0 .` চালান — শেষের `.` ভুলে গেলে build context পাবে না, এটাই সবচেয়ে সাধারণ ভুল।

ধাপ 4: Build log-এ প্রতিটি ধাপের layer ID দেখুন, তারপর `docker images technova-site` দিয়ে আকার দেখুন।

ধাপ 5: `docker run -d --name my-site -p 8080:80 technova-site:1.0` চালিয়ে `curl http://localhost:8080` -এ নিজের HTML দেখুন।

ধাপ 6: `index.html` বদলে আবার build করুন — শুধু COPY layer বদলায়, FROM layer cache থেকে আসে (ল্যাব ৯-এর ভূমিকা)।

ধাপ 7: `docker history technova-site:1.0` দিয়ে দেখুন আপনার COPY একটি আলাদা layer হয়েছে।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker images technova-site
curl -s http://localhost:8080 | grep -i technova
docker history technova-site:1.0
# প্রত্যাশিত: image তৈরি হয়, curl-এ আপনার নিজের HTML আসে,
# history-তে COPY layer দেখা যায়।
```

প্রত্যাশিত ফলাফল: নিজের Dockerfile থেকে image তৈরি হয়, সেটি থেকে container চলে, ব্রাউজারে নিজের HTML দেখা যায়। Build log-এ প্রতিটি নির্দেশ একটি layer — এই ধারণাই পরের ল্যাবগুলোর ভিত্তি।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
Build-এ `failed to compute cache key: index.html not found`	Dockerfile যে ফোল্ডারে, `index.html` -ও সেই ফোল্ডারে থাকতে হবে — build context-এর বাইরের ফাইল COPY করা যায় না।
`COPY failed` বা পথ ভুল	`COPY index.html ./` -এ source হলো context-এর সাপেক্ষে পথ, হোস্টের absolute পথ নয়।
Container চলে কিন্তু পুরনো nginx পাতা দেখাচ্ছে	`WORKDIR` ভুল — nginx-এর ডিফল্ট html ফোল্ডার `/usr/share/nginx/html` ; অন্য ফোল্ডারে কপি করলে nginx সেটি দেখাবে না।
CMD লিখেও container থামছে	`CMD ["nginx", "-g", "daemon off;"]` ঠিক থাকতে হবে — nginx daemon mode-এ গেলে মূল প্রসেস শেষ হয়ে container থেমে যায়।

Troubleshooting

Build ব্যর্থ হলে log-এর প্রথম লাল লাইনটি পড়ুন — Docker সাধারণত ঠিক কোন ধাপে ও কেন ব্যর্থ হয়েছে তা বলে দেয়। `docker build --no-cache -t ... .` দিয়ে cache ছাড়া build করে দেখুন সমস্যাটি cache-জনিত কি না। পথ নিয়ে সন্দেহ হলে `ls -la` দিয়ে context ফোল্ডারের ভেতরটা দেখুন।

Cleanup

`docker stop my-site && docker rm my-site`; image রাখুন, ল্যাব ৯-এ লাগবে। ফোল্ডার `~/technova-lab7/` রেখে দিন।

Reflection প্রশ্ন

১. `RUN` বনাম `CMD` বনাম `ENTRYPOINT` — কোনটি build-এর সময় চলে, কোনটি container চালুর সময়? ২. Build context বড় হলে build ধীর হয় কেন — `.dockerignore` কীভাবে সাহায্য করে? ৩. `EXPOSE 80` না লিখলে কি port mapping কাজ করত না?

ল্যাব ৪

Node.js অ্যাপ্লিকেশন Containerize করা

উদ্দেশ্য (Objective): TechNova-র ন্যূনতম Express অ্যাপের জন্য Dockerfile লেখা — dependency install, port, `.dockerignore` — এবং container থেকে API চালানো। এই ল্যাবটি পুরো বইয়ের মূল কাজ: আপনার নিজের কোডকে portable image-এ পরিণত করা।

পূর্বশর্ত (Prerequisites): ল্যাব ৭ সম্পূর্ণ; Node.js ও Express-এর ধারণা আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, Node.js 20 (হোস্টে পরীক্ষার জন্য), `~/technova-app/` ফোল্ডার।

আর্কিটেকচার:
Host কোড (server/ + package.json) → docker build → Image: technova-api:1.0 (node:20-alpine + node_modules + server.js) → Container (port 3000) → curl localhost:3000/health → {"status":"ok"}

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# ~/technova-app/server/package.json
{
  "name": "technova-api",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": { "start": "node server.js" },
  "dependencies": { "express": "^4.19.2" }
}

# ~/technova-app/server/server.js
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;
app.get('/health', (req, res) => res.json({ status: 'ok', app: 'technova-api' }));
app.get('/', (req, res) => res.send('Hello from TechNova API (container)!'));
app.listen(PORT, '0.0.0.0', () => console.log(`API listening on ${PORT}`));

# ~/technova-app/server/Dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . ./
EXPOSE 3000
CMD ["node", "server.js"]

# ~/technova-app/server/.dockerignore
node_modules
npm-debug.log
.git
.env

# build ও run
```

```
docker build -t technova-api:1.0 ./server
docker run -d --name technova-api -p 3000:3000 technova-api:1.0
curl -s http://localhost:3000/health
# নমুনা আউটপুট — চিত্রণমূলক: {"status":"ok","app":"technova-api"}
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `~/technova-app/server/` ফোল্ডারে উপরের `package.json` ও `server.js` লিখুন; হোস্টে `npm install && node server.js` দিয়ে প্রথমে Docker ছাড়া চালিয়ে দেখুন API সাড়া দেয় কি না।

ধাপ 2: Dockerfile লিখুন — লক্ষ্য করুন `COPY package*.json` আগে, `RUN npm ci` তারপর, `COPY . ./` শেষে — এই ক্রমই cache বাঁচায় (ল্যাব ৯)।

ধাপ 3: `.dockerignore` লিখুন — `node_modules` ও `.env` image-এ যাবে না; `.env` image-এ গেলে secret ফাঁস হয়।

ধাপ 4: `docker build -t technova-api:1.0 ./server` চালান (context `./server`, Dockerfile তার ভেতরে)।

ধাপ 5: `docker run -d --name technova-api -p 3000:3000 technova-api:1.0` চালিয়ে `curl http://localhost:3000/health` ও `curl http://localhost:3000/` পরীক্ষা করুন।

ধাপ 6: `docker logs technova-api` দিয়ে "API listening on 3000" দেখুন; `docker exec -it technova-api sh` দিয়ে ভেতরে `ls /app` দেখুন — `node_modules` আছে, কিন্তু হোস্টের `.env` নেই।

ধাপ 7: `server.js`-এ লেখা বদলে আবার build ও run করুন — নতুন image-এ নতুন লেখা আসে কি না দেখুন।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
curl -s http://localhost:3000/health
docker logs technova-api | grep -i listening
docker exec technova-api ls /app
# প্রত্যাশিত: {"status":"ok","app":"technova-api"}, log-এ listening লাইন,
# /app-এ server.js, package.json ও node_modules আছে।
```

প্রত্যাশিত ফলাফল: নিজের Express অ্যাপ image থেকে চলে, `/health` JSON দেয়, log-এ listening লাইন আসে; `.dockerignore`-এর কারণে হোস্টের `node_modules` ও `.env` image-এ যায়নি।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
`npm ci` ব্যর্থ: lockfile নেই	প্রথমবার হোস্টে `npm install` করে `package-lock.json` তৈরি করুন, তারপর build করুন — `npm ci`-এর lockfile লাগে।
curl-এ connection refused, কিন্তু container Up	অ্যাপ `app.listen(PORT, '0.0.0.0')` দিয়ে শুনছে কি না দেখুন — শুধু `localhost`-এ শুনলে container-এর বাইরে থেকে পৌঁছানো যায় না।

Build context-এ `.env` চলে গেছে	`.dockerignore` Dockerfile-এর পাশের ফোল্ডারে (context root-এ) আছে কি না দেখুন; না থাকলে secret image-এ ঢুকে যায়।
Port 3000 ব্যস্ত	হোস্টে Node.js সরাসরি চলছে কি না দেখুন — হোস্টের অ্যাপ থামান বা অন্য port map করুন: `-p 3001:3000`।

Troubleshooting

API সাড়া না দিলে `docker logs technova-api` প্রথমে দেখুন — Express-এর error (যেমন port ব্যস্ত, module পাওয়া যায়নি) সেখানেই থাকে। তারপর `docker exec -it technova-api sh` দিয়ে ভেতরে `node --version` ও `ls /app` যাচাই করুন। হোস্টে চলা অ্যাপ আর container-এর অ্যাপ একই port-এ থাকলে হোস্টেরটি থামাতে ভুলবেন না।

Cleanup

Container থামিয়ে মুছুন: `docker stop technova-api && docker rm technova-api`; image ও কোড রাখুন — ল্যাব ৯–১১ এগুলোর উপরেই হবে।

Reflection প্রশ্ন

১. `npm ci` বনাম `npm install` — image build-এ কোনটি ভালো, এবং কেন? ২. `.dockerignore` না থাকলে কী কী ঝুঁকি (আকার, secret, গতি)? ৩. অ্যাপকে `0.0.0.0`-এ শুনতে হয় কেন — `localhost`-এ শুনলে কী হয়?

ল্যাব 9

Build Cache নিয়ে পরীক্ষা করা

উদ্দেশ্য (Objective): Docker build cache কীভাবে কাজ করে তা পরীক্ষা করা — layer-এর ক্রম বদলে, ফাইল বদলে ও `--no-cache` দিয়ে build সময় মাপা। এই ল্যাবের পর আপনি Dockerfile-এর নির্দেশ সাজিয়ে build সময় কয়েক গুণ কমাতে পারবেন।

পূর্বশর্ত (Prerequisites): ল্যাব ৮ সম্পূর্ণ; technova-api কোড ও Dockerfile আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, technova-api প্রজেক্ট, ঘড়ি (build সময় মাপার জন্য)।

আর্কিটেকচার:
Dockerfile ধাপ: FROM (cache) → COPY package.json (বদলালে cache ভাঙে) → RUN npm ci (সবচেয়ে ধীর ধাপ) → COPY . ./ (কোড বদলালে শুধু এই layer ও পরেরগুলো আবার বানায়)

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# প্রথম build (cold): সময় মাপুন
cd ~/technova-app/server
time docker build -t technova-api:cache-test .
# নমুনা আউটপুট — চিত্রণমূলক: real 0m45.123s (প্রথমবার npm ci ধীর)

# কিছু না বদলে আবার build: সব layer cache থেকে আসবে
time docker build -t technova-api:cache-test .
# নমুনা আউটপুট — চিত্রণমূলক: real 0m1.234s, প্রতিটি ধাপে CACHED

# শুধু server.js-এ একটি মন্তব্য বদলান, আবার build করুন
echo "// cache test comment" >> server.js
time docker build -t technova-api:cache-test .
# প্রত্যাশিত: COPY package*.json ও npm ci CACHED, শুধু COPY . ./ আবার চলে

# cache ছাড়া build
time docker build --no-cache -t technova-api:cache-test .

# কোন layer কত বড় দেখুন
docker history technova-api:cache-test
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: কিছু না বদলে দুবার build করে সময় লিখুন — প্রথমবার (cold) বনাম দ্বিতীয়বার (সব CACHED)।

ধাপ 2: শুধু `server.js`-এ একটি মন্তব্য যোগ করে build করুন — log-এ দেখুন `npm ci` ধাপটি CACHED থাকে, শুধু শেষের COPY আবার চলে।

ধাপ 3: এবার `package.json`-এ একটি নির্ভরতা (যেমন সংস্করণ) বদলে build করুন — দেখুন `npm ci` আবার চলে, কারণ তার আগের COPY layer বদলে গেছে।

ধাপ 4: `docker build --no-cache` দিয়ে সম্পূর্ণ নতুন build-এর সময় মাপুন — cache কত সময় বাঁচায় তা সংখ্যায় লিখুন।

ধাপ 5: পরীক্ষা করুন: Dockerfile-এ `COPY . ./` লাইনটি `RUN npm ci`-এর আগে সরিয়ে (ভুল ক্রমে) build করুন — প্রতিবার কোড বদলালে `npm ci` আবার চলবে; তারপর সঠিক ক্রমে ফিরিয়ে আনুন।

ধাপ 6: `docker history` দিয়ে সবচেয়ে বড় ও ধীর layer চিহ্নিত করুন — সাধারণত `npm ci` layer।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker build -t technova-api:cache-test . 2>&1 | grep -i cached | wc -l
# প্রত্যাশিত: দ্বিতীয় build-এ প্রায় সব ধাপ CACHED, সময় 1–2 সেকেন্ড
docker history technova-api:cache-test --format '{{.CreatedBy}} {{.Size}}'
```

প্রত্যাশিত ফলাফল: দ্বিতীয় build কয়েক সেকেন্ডে শেষ হয় (সব CACHED); শুধু কোড বদলালে `npm ci` আবার চলে না; `package.json` বদলালে `npm ci` আবার চলে; ভুল ক্রমে COPY আগে থাকলে প্রতিবার `npm ci` আবার চলে — cache-এর নিয়মটি সংখ্যায় প্রমাণিত হয়।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
প্রতিবার build ধীর, CACHED দেখাচ্ছে না	BuildKit বন্ধ কি না দেখুন — `DOCKER_BUILDKIT=1 docker build ...` দিয়ে চেষ্টা করুন; অথবা build context-এ `node_modules` ঢুকে যাচ্ছে কি না `.dockerignore` দেখুন।
ফাইল বদলাইনি, তবু cache ভাঙছে	`COPY . ./`-এর context-এ কোনো ফাইল (যেমন log) বদলাচ্ছে কি না দেখুন — context-এর যেকোনো ফাইল বদলালে COPY layer-এর cache ভাঙে।
Cache আছে, তবু `npm ci` চলছে	`package.json` বা `package-lock.json`-এর timestamp/content বদলেছে — `git status` দিয়ে দেখুন কী বদলেছে।

Troubleshooting

Cache আচরণ বুঝতে `docker build --progress=plain` ব্যবহার করুন — প্রতিটি ধাপে CACHED লেখা স্পষ্ট দেখা যায়। সন্দেহ হলে একটি ছোট পরীক্ষা করুন: একটি ফাইল বদলান, build করুন, log-এ কোন ধাপ থেকে CACHED থেমে গেছে তা দেখুন — সেখানেই cache ভেঙেছে।

Cleanup

`docker rmi technova-api:cache-test` মুছে ফেলুন; মূল `technova-api:1.0` রাখুন। `server.js`-এর পরীক্ষামূলক মন্তব্য রেখে দিতে পারেন বা মুছে ফেলুন — পরের ল্যাবে নতুন Dockerfile লেখা হবে।

Reflection প্রশ্ন

১. Cache ভাঙে কোন নিয়মে — একটি layer বদলালে তার পরের সব layer কেন আবার বানাতে হয়? ২. `COPY package*.json` আগে রাখার সুবিধা কী — কোড বদলালে dependency install কেন আবার হয় না? ৩. CI-তে cache কীভাবে বাঁচানো যায় (layer cache), আর `--no-cache` কখন দরকার?

ল্যাব 10

Multi-stage Dockerfile তৈরি করা

উদ্দেশ্য (Objective): Multi-stage build লেখা — build stage-এ dependency install ও frontend build, runtime stage-এ শুধু চলার জন্য দরকারি ফাইল — যাতে চূড়ান্ত image ছোট ও নিরাপদ হয়। এই ল্যাবের পর আপনার image-এ build tool (npm, git, compiler) থাকবে না।

পূর্বশর্ত (Prerequisites): ল্যাব ৮–৯ সম্পূর্ণ; technova-api আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, technova-api প্রজেক্ট, React/Vite frontend থাকলে আরও ভালো (না থাকলে API দিয়েই করা যাবে)।

আর্কিটেকচার:
Stage 1 (build): node:20 → npm ci → npm run build → dist/ | Stage 2 (runtime): node:20-alpine → COPY --from=build /app/dist → শুধু production ফাইল → ছোট, নিরাপদ image

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# ~/technova-app/server/Dockerfile.multistage
# ---------- Stage 1: build ----------
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . ./
RUN npm run build 2>/dev/null || echo "no build script - skipping"
# (API-র ক্ষেত্রে build script না থাকলে এই ধাপটি শুধু প্রমাণ হিসেবে চলে)

# ---------- Stage 2: runtime ----------
FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=build /app/server.js ./server.js
EXPOSE 3000
USER node
CMD ["node", "server.js"]

# build করুন (target নির্দিষ্ট না করলে শেষ stage-ই চূড়ান্ত image)
docker build -f Dockerfile.multistage -t technova-api:multi .
docker images technova-api
# শুধু build stage-এর image দেখতে (পরীক্ষার জন্য):
docker build -f Dockerfile.multistage --target build -t technova-api:build-stage .
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: উপরের `Dockerfile.multistage` লিখুন — দুটি `FROM` দেখুন: প্রথমটির নাম `build`, দ্বিতীয়টির নাম `runtime`।

ধাপ 2: `docker build -f Dockerfile.multistage -t technova-api:multi .` চালিয়ে build log-এ দুটি stage আলাদা করে দেখুন।

ধাপ 3: `docker images technova-api` দিয়ে single-stage (`1.0`) ও multi-stage (`multi`) image-এর আকার পাশাপাশি লিখুন।

ধাপ 4: `docker run -d --name api-multi -p 3001:3000 technova-api:multi` চালিয়ে `curl http://localhost:3001/health` পরীক্ষা করুন — ছোট image-ও একই API দেয় কি না।

ধাপ 5: `docker exec -it api-multi sh` দিয়ে ভেতরে দেখুন: `npm` আছে কি? `node_modules/.bin`-এ dev tool আছে কি? (থাকার কথা নয় — `--omit=dev` দেওয়া হয়েছে।)

ধাপ 6: `--target build` দিয়ে শুধু build stage-এর image বানিয়ে তার আকার দেখুন — runtime stage কতটা ছোট তা সংখ্যায় বুঝুন।

ধাপ 7: Frontend থাকলে একই কৌশলে React build (Vite) stage থেকে `dist/` নিয়ে nginx stage-এ পরিবেশন করুন — ল্যাব ১৮-এ এটিই পূর্ণ হবে।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker images technova-api --format 'table {{.Tag}}\t{{.Size}}'
curl -s http://localhost:3001/health
docker exec api-multi sh -c 'ls /app && whoami'
# প্রত্যাশিত: multi image single-stage-এর চেয়ে ছোট; /health ok;
# ভেতরে user node (root নয়)।
```

প্রত্যাশিত ফলাফল: Multi-stage image single-stage-এর চেয়ে ছোট, একই `/health` দেয়, ভেতরে dev dependency ও build tool নেই, container `node` user-এ চলে। Build stage-এর image আলাদা করে বানালে দেখা যায় runtime stage কতটা হালকা।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
COPY --from=build-এ ফাইল পাওয়া যাচ্ছে না	Stage-এর নাম (`build`) ও পথ (`/app/server.js`) ঠিক আছে কি না দেখুন — build stage-এ WORKDIR `/app` ছিল।
Build stage-এ `npm run build` ব্যর্থ	API-তে build script নেই — ল্যাবের Dockerfile-এ `|| echo` দিয়ে সেটি সামলানো আছে; frontend থাকলে আসল build script লিখুন।
Multi-stage image আশানুরূপ ছোট হয়নি	Runtime stage-এও `npm ci` (dev ছাড়া) চলছে কি না, আর `node:20` (পূর্ণ) না `node:20-alpine` ব্যবহার করছেন কি না দেখুন।
USER node-এ permission denied	`COPY` করা ফাইল root-এর মালিকানায় থাকে — ল্যাব ২১-এ `--chown=node:node` দিয়ে মালিকানা ঠিক করা শিখবেন।

Troubleshooting

Multi-stage build ব্যর্থ হলে `--target build` দিয়ে শুধু প্রথম stage বানিয়ে দেখুন — সমস্যাটি build stage-এ না runtime stage-এ তা আলাদা হয়ে যাবে। `docker build --progress=plain` দিয়ে কোন stage-এ থেমেছে দেখুন।

Cleanup

`docker stop api-multi && docker rm api-multi; docker rmi technova-api:build-stage; technova-api:multi` রাখুন — প্রজেক্ট ১-এর ভিত্তি এটাই।

Reflection প্রশ্ন

১. Build tool image-এ থাকলে আক্রমণের সুযোগ (attack surface) কীভাবে বাড়ে? ২. `COPY --from=build` ছাড়া আর কীভাবে stage-এর মধ্যে ফাইল নেওয়া যায়? ৩. কখন multi-stage দরকার নেই — একটিমাত্র ছোট স্ক্রিপ্টের ক্ষেত্রে কী করবেন?

ল্যাব 11

Image আকার Optimize করা

উদ্দেশ্য (Objective): একই অ্যাপের image আকার কমানো — alpine base, `--omit=dev`, layer একত্র করা, `.dockerignore`, cache পরিষ্কার — এবং প্রতিটি কৌশলের প্রভাব সংখ্যায় মাপা। এই ল্যাবের পর আপনি যেকোনো image-কে অর্ধেক বা তারও কম আকারে আনতে পারবেন।

পূর্বশর্ত (Prerequisites): ল্যাব ১০ সম্পূর্ণ; technova-api:1.0 ও :multi আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, technova-api প্রজেক্ট, `docker images` ও `docker history`।

আর্কিটেকচার:
তুলনা টেবিল (একই অ্যাপ): node:20 (পূর্ণ, ~1 GB) → node:20-slim (~200 MB) → node:20-alpine (~170 MB) → multi-stage alpine + --omit=dev (~120 MB) — সংখ্যা আপনার সিস্টেমে আলাদা হতে পারে, কিন্তু ক্রম একই থাকবে

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# ভিন্ন base image-এ একই Dockerfile build করে তুলনা করুন
# (Dockerfile-এর FROM লাইনটি বদলে বদলে build করুন)
docker build -t technova-api:full -f Dockerfile.full .     # FROM node:20
docker build -t technova-api:slim -f Dockerfile.slim .     # FROM node:20-slim
docker build -t technova-api:alpine -f Dockerfile.multistage .  # alpine runtime
docker images technova-api --format 'table {{.Tag}}\t{{.Size}}'

# layer একত্র করা: আলাদা RUN-এর বদলে এক RUN-এ install + clean
# খারাপ (দুটি layer, cache-এ apt ফাইল থেকে যায়):
#   RUN apt-get update
#   RUN apt-get install -y curl
# ভালো (এক layer, cache মুছে ফেলা হয়):
#   RUN apt-get update && apt-get install -y --no-install-recommends curl \
#       && rm -rf /var/lib/apt/lists/*

# npm cache পরিষ্কার করা
# RUN npm ci --omit=dev && npm cache clean --force

# আকারের হিসাব দেখুন
docker system df
docker history technova-api:alpine --human | head -15
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: তিনটি base image-এ (`node:20`, `node:20-slim`, `node:20-alpine`) একই অ্যাপ build করে আকারের টেবিল লিখুন — পার্থক্য কত MB, কত শতাংশ?

ধাপ 2: `docker history --human` দিয়ে সবচেয়ে বড় layer খুঁজুন — সেটি base image, না `npm ci` layer?

ধাপ 3: `.dockerignore`-এ `node_modules`, `.git`, `*.md`, `.env` যোগ করে build করুন — build context-এর আকার (`docker build`-এর প্রথম লাইনে "transferring context" দেখায়) কত কমল তা লিখুন।

ধাপ 4: দুটি RUN আলাদা লেখা Dockerfile বনাম এক RUN-এ `&&` দিয়ে লেখা Dockerfile — দুটির আকার তুলনা করুন; cache ফাইল মুছে ফেলার (`rm -rf /var/lib/apt/lists/*`) প্রভাব দেখুন।

ধাপ 5: `npm ci --omit=dev` বনাম `npm ci` (সব dependency) — দুটি image-এর আকার তুলনা করুন।

ধাপ 6: সবচেয়ে ছোট image-টি চালিয়ে `curl /health` দিয়ে যাচাই করুন — ছোট image মানেই ভাঙা image নয়, তা প্রমাণ করুন।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker images technova-api --format 'table {{.Tag}}\t{{.Size}}'
docker system df
# প্রত্যাশিত: full > slim > alpine; multi-stage alpine সবচেয়ে ছোট;
# প্রতিটি কৌশলের পর আকার কমে।
```

প্রত্যাশিত ফলাফল: একই অ্যাপের image base image ও কৌশলভেদে কয়েক শ গুণ পর্যন্ত ছোট হয়; সবচেয়ে ছোট multi-stage alpine image-ও একই API দেয়; `docker system df`-তে মোট disk ব্যবহার কমে। আপনার নোটে প্রতিটি কৌশলের আগে-পরের আকার থাকবে।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
alpine-এ native module build ব্যর্থ	কিছু npm প্যাকেজের C compiler লাগে — alpine-এ `apk add --no-cache python3 make g++` build stage-এ দিন, runtime stage-এ নয়।
slim image-এ `curl` নেই	slim/alpine-এ অনেক টুল থাকে না — health check-এর জন্য `wget` বা Node.js-এর নিজের HTTP ব্যবহার করুন (ল্যাব ২০)।
আকার কমছে না	`docker history` দিয়ে দেখুন কোন layer বড় — অনেক সময় base image-ই সবচেয়ে বড়, তখন base বদলানোই একমাত্র উপায়।
Context transfer ধীর	`.dockerignore` ঠিক আছে কি না দেখুন — `node_modules` context-এ ঢুকলে প্রতিবার MB-খানেক পাঠাতে হয়।

Troubleshooting

আকার না কমলে `docker history --human --no-trunc` দিয়ে layer ধরে ধরে দেখুন — অনুমান নয়, সংখ্যাই বলবে কোন layer দায়ী। `dive` টুল (ঐচ্ছিক) দিয়ে layer-এর ভেতরের ফাইলও দেখা যায়। মনে রাখুন: আকার কমানো উদ্দেশ্য, কিন্তু image-এর কাজ (API সাড়া দেওয়া) ঠিক থাকতে হবে — প্রতিটি optimization-এর পর `curl /health` দিয়ে যাচাই করুন।

Cleanup

পরীক্ষামূলক tag-গুলো (`full`, `slim`) মুছে ফেলুন: `docker rmi technova-api:full technova-api:slim`; সবচেয়ে ছোটটি (`multi`/alpine) রাখুন।

Reflection প্রশ্ন

১. ছোট image কেন শুধু disk নয়, pull সময়, deploy সময় ও নিরাপত্তা — তিনটিতেই জেতে? ২. `alpine` কখন ঝুঁকিপূর্ণ (musl বনাম glibc) — কোন অ্যাপে পূর্ণ base ভালো? ৩. Layer একত্র করা কখন cache-এর ক্ষতি করে — trade-off কী?

ল্যাব 12

Image Docker Hub-এ Publish করা

উদ্দেশ্য (Objective): নিজের image-কে tag করে Docker Hub-এ push করা, অন্য মেশিন থেকে pull করে চালানো, আর private/public-এর পার্থক্য বোঝা। এই ল্যাবের পর আপনার image যেকোনো Docker থাকা মেশিনে চালানো যাবে।

পূর্বশর্ত (Prerequisites): ল্যাব ১১ সম্পূর্ণ; technova-api:multi আছে; Docker Hub অ্যাকাউন্ট আছে (বিনামূল্যে খোলা যায়)।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, Docker Hub অ্যাকাউন্ট, ইন্টারনেট, technova-api:multi image।

আর্কিটেকচার:
Local image technova-api:multi → docker tag → yourname/technova-api:1.0 → docker push → Docker Hub registry (public/private) → অন্য মেশিন: docker pull yourname/technova-api:1.0 → docker run

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# Docker Hub-এ login করুন (পাসওয়ার্ড টার্মিনালে টাইপ করবেন না — prompt-এ দেবেন)
docker login
# নমুনা আউটপুট — চিত্রণমূলক: Login Succeeded

# নিজের Docker Hub username দিয়ে tag করুন (yourname-এর জায়গায় আসল username)
docker tag technova-api:multi yourname/technova-api:1.0
docker tag technova-api:multi yourname/technova-api:latest

# push করুন
docker push yourname/technova-api:1.0
docker push yourname/technova-api:latest
# নমুনা আউটপুট — চিত্রণমূলক: 1.0: digest: sha256:ab12... size: 1234

# যাচাই: স্থানীয় image মুছে registry থেকে আবার নামান
docker rmi yourname/technova-api:1.0
docker pull yourname/technova-api:1.0
docker run -d --name hub-test -p 3002:3000 yourname/technova-api:1.0
curl -s http://localhost:3002/health
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: Docker Hub-এ অ্যাকাউন্ট খুলুন (hub.docker.com) — repository-টি প্রথমে private রাখুন, যাতে পরীক্ষামূলক image সবাই দেখতে না পায়।

ধাপ 2: `docker login` চালিয়ে username ও password দিন — password কখনো কমান্ড লাইনে লিখবেন না, কারণ shell history-তে থেকে যাবে।

ধাপ 3: `docker tag technova-api:multi yourname/technova-api:1.0` দিয়ে registry-র নামে tag করুন — `yourname`-এর জায়গায় আপনার Docker Hub username বসবে।

ধাপ 4: `docker push yourname/technova-api:1.0` চালান; log-এ digest লিখে রাখুন — digest-ই প্রমাণ করে কোন image push হয়েছে।

ধাপ 5: Hub-এর ওয়েবসাইটে গিয়ে repository-টি দেখুন — tag, আকার ও তৈরির সময় মিলিয়ে দেখুন।

ধাপ 6: স্থানীয় tag মুছে `docker pull yourname/technova-api:1.0` দিয়ে আবার নামান, চালিয়ে `curl /health` পরীক্ষা করুন — registry থেকে আসা image-ও একইভাবে চলে।

ধাপ 7: একটি ভুল tag (যেমন `:v9.9.9`) push করে দেখুন Hub-এ সেটি কীভাবে দেখায়, তারপর Hub UI থেকে tag-টি মুছে ফেলুন।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker pull yourname/technova-api:1.0
curl -s http://localhost:3002/health
# প্রত্যাশিত: {"status":"ok","app":"technova-api"} — registry থেকে নামানো image চলে।
docker logout
# logout-এর পরেও pull করা public image চলে, কিন্তু push করা যায় না।
```

প্রত্যাশিত ফলাফল: Image Docker Hub-এ push হয়, ওয়েবসাইটে tag ও digest দেখা যায়, স্থানীয় কপি মুছে pull করে আবার চালানো যায় — একই API সাড়া দেয়। Login ছাড়া push হয় না, এটাও প্রমাণিত হয়।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
Push-এ `denied: requested access to the resource is denied`	Login হয়নি বা username ভুল — `docker login` আবার করুন; tag-এ username ঠিক আছে কি না দেখুন।
Push-এ `name invalid`	Repository নাম ছোট হাতের অক্ষরে হতে হবে — বড় হাতের অক্ষর বা space থাকলে এমন হয়।
Private repository pull-এ `pull access denied`	Private image pull-এ login লাগে — `docker login` করে আবার pull করুন।
Push খুব ধীর	প্রথম push-এ সব layer যায়; পরের push-এ শুধু বদলানো layer যায় — image ছোট রাখার (ল্যাব ১১) আরেকটি সুবিধা।

Troubleshooting

Push ব্যর্থ হলে `docker login`-এর অবস্থা (`cat ~/.docker/config.json`-এ auth আছে কি না — password দেখা যাবে না, শুধু auth entry দেখা যাবে) ও tag-এর নাম মিলিয়ে দেখুন। Registry-র সমস্যা হলে Hub-এর status পাতা দেখুন — অনেক সময় সমস্যা আপনার মেশিনে নয়, registry-তে।

Cleanup

`docker stop hub-test && docker rm hub-test; docker logout`। Hub-এর পরীক্ষামূলক repository চাইলে private রেখে দিন বা মুছে ফেলুন — public-এ ভুল image রেখে দেবেন না।

Reflection প্রশ্ন

১. Public বনাম private repository — কখন কোনটি? Secret কি কখনো image-এ push করা যায়? ২. Tag push করলেই কি পুরো image আবার যায়, নাকি শুধু বদলানো layer? ৩. প্রোডাকশনে `:latest` pull করা কেন বিপজ্জনক — digest দিয়ে pull করা কেন ভালো?

ল্যাব 13

Custom Bridge Network তৈরি করা

উদ্দেশ্য (Objective): Docker-এর ডিফল্ট bridge-এর বদলে নিজের bridge network বানানো — subnet, gateway ও DNS সুবিধাসহ — এবং container-কে সেটিতে যুক্ত করা। এই ল্যাবের পর আপনি প্রতিটি অ্যাপের জন্য আলাদা, বিচ্ছিন্ন নেটওয়ার্ক বানাতে পারবেন।

পূর্বশর্ত (Prerequisites): ল্যাব ৪ সম্পূর্ণ; network-এর ধারণা আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, nginx ও alpine image।

আর্কিটেকচার:
Host → docker network technova-net (bridge, 172.25.0.0/16) → Container A (172.25.0.2) ↔ Container B (172.25.0.3) — একই network-এর container-রা একে অপরকে নামে ডাকতে পারে; অন্য network-এর container পারে না

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# নিজের bridge network বানান (subnet নিজে ঠিক করুন)
docker network create --driver bridge --subnet 172.25.0.0/16 --gateway 172.25.0.1 technova-net

# network-গুলো দেখুন ও বিস্তারিত দেখুন
docker network ls
docker network inspect technova-net --format 'Subnet: {{(index .IPAM.Config 0).Subnet}}
Gateway: {{(index .IPAM.Config 0).Gateway}}'

# দুটি container এই network-এ চালান
docker run -d --name web-a --network technova-net nginx:1.27-alpine
docker run -d --name web-b --network technova-net nginx:1.27-alpine

# ডিফল্ট bridge-এ একটি container চালান (তুলনার জন্য)
docker run -d --name web-default nginx:1.27-alpine

# network-এ কে কে আছে দেখুন
docker network inspect technova-net --format '{{range .Containers}}{{.Name}} {{.IPv4Address}}
{{end}}'
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `docker network ls` দিয়ে ডিফল্ট network-গুলো (`bridge`, `host`, `none`) দেখুন ও লিখুন।

ধাপ 2: `docker network create --driver bridge --subnet 172.25.0.0/16 technova-net` চালিয়ে নিজের network বানান — subnet হোস্টের LAN-এর সঙ্গে না মিলিয়ে বেছে নিন।

ধাপ 3: দুটি nginx container `--network technova-net` দিয়ে চালান, আরেকটি ডিফল্ট bridge-এ চালান।

ধাপ 4: `docker network inspect technova-net` দিয়ে প্রতিটি container-এর IP লিখুন — IP-গুলো 172.25.0.x পরিসরে কি না দেখুন।

ধাপ 5: একটি container থামিয়ে আবার চালান — IP বদলায় কি না দেখুন (নাম স্থির থাকে, IP বদলাতে পারে — তাই নামে ডাকাই সঠিক পদ্ধতি)।

ধাপ 6: `docker network connect` দিয়ে একটি container-কে দ্বিতীয় network-এ যুক্ত করুন, তারপর `docker network disconnect` দিয়ে সরান।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker network ls
docker network inspect technova-net --format '{{range .Containers}}{{.Name}} {{.IPv4Address}}
{{end}}'
# প্রত্যাশিত: technova-net-এ web-a ও web-b, IP 172.25.0.x পরিসরে।
# নমুনা আউটপুট — চিত্রণমূলক: web-a 172.25.0.2/16 web-b 172.25.0.3/16
```

প্রত্যাশিত ফলাফল: নিজের bridge network তৈরি হয়, দুটি container তাতে IP পায়, `inspect`-এ subnet ও সদস্য দেখা যায়; ডিফল্ট bridge-এর container আলাদা subnet-এ থাকে।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
Subnet ইতিমধ্যে ব্যবহৃত হচ্ছে	অন্য কোনো network বা হোস্টের LAN একই subnet ব্যবহার করছে — ভিন্ন subnet বেছে নিন, যেমন 172.26.0.0/16।
Network তৈরি হচ্ছে না: `already exists`	আগের ল্যাবের network আছে — `docker network ls` দেখে নাম বদলান বা পুরনোটি মুছুন।
Container-এর IP প্রত্যাশিত পরিসরে নেই	Container ভুল network-এ চালু হয়েছে কি না `docker inspect <name> --format '{{.NetworkSettings.Networks}}'` দিয়ে দেখুন।

Troubleshooting

Network সমস্যা হলে `docker network inspect`-এর `Containers` ও `IPAM.Config` অংশ দেখুন — কোন container কোন network-এ, কোন IP-তে, তা সেখানেই লেখা থাকে। `docker network prune` চালানোর আগে দেখুন কোনো ব্যবহৃত network মুছে যাচ্ছে কি না — ব্যবহৃত network prune হয় না, তবে নাম ভুলে অন্য network মুছে ফেলার ঝুঁকি থাকে।

Cleanup

`docker stop web-a web-b web-default && docker rm web-a web-b web-default; docker network rm technova-net` — network মুছতে হলে আগে তার সব container সরাতে হয়।

Reflection প্রশ্ন

১. Custom bridge বনাম ডিফল্ট bridge — DNS-এর পার্থক্য কী (ল্যাব ১৫-এ প্রমাণ হবে)? ২. Subnet নিজে ঠিক করা কেন দরকার — হোস্টের LAN-এর সঙ্গে মিলে গেলে কী হয়? ৩. একটি container দুটি network-এ থাকলে কী সুবিধা হয়?

ল্যাব 14

Container-to-Container যোগাযোগ পরীক্ষা করা

উদ্দেশ্য (Objective): একই network-এর দুটি container-এর মধ্যে HTTP request পাঠানো — নামে ও IP-তে — এবং ভিন্ন network-এর container-এর সঙ্গে যোগাযোগ বন্ধ থাকা প্রমাণ করা। এই ল্যাবটি microservice-এর মূল: সার্ভিসগুলো একে অপরকে কীভাবে ডাকে।

পূর্বশর্ত (Prerequisites): ল্যাব ১৩ সম্পূর্ণ; technova-net বানাতে পারেন।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, technova-net, nginx ও technova-api image।

আর্কিটেকচার:
technova-net: api (technova-api:3000) ← web (nginx) : curl http://api:3000/health ✅ | অন্য network-এর container → api: ❌ (পৌঁছায় না) | Host → api: শুধু port publish করলে ✅

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# network ও দুটি container বানান
docker network create technova-net
docker run -d --name api --network technova-net technova-api:1.0
docker run -d --name web --network technova-net nginx:1.27-alpine

# web container-এর ভেতর থেকে api-কে নামে ডাকুন
docker exec web sh -c 'wget -qO- http://api:3000/health || curl -s http://api:3000/health'
# নমুনা আউটপুট — চিত্রণমূলক: {"status":"ok","app":"technova-api"}

# IP দিয়েও পরীক্ষা করুন
docker exec web sh -c 'wget -qO- http://172.25.0.2:3000/health' || true
# (IP আপনার সিস্টেমে আলাদা হবে — inspect দিয়ে আসল IP নিন)

# ভিন্ন network-এর container থেকে চেষ্টা করুন (ব্যর্থ হওয়ার কথা)
docker run -d --name outsider nginx:1.27-alpine
docker exec outsider sh -c 'wget -qO- --timeout=3 http://api:3000/health' || echo "EXPECTED
FAILURE: outsider cannot reach api"
# নমুনা আউটপুট — চিত্রণমূলক: EXPECTED FAILURE
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `technova-net` বানিয়ে `api` (technova-api) ও `web` (nginx) container দুটি তাতে চালান — port publish করবেন না, শুধু network-এর ভেতরের যোগাযোগ পরীক্ষা করবেন।

ধাপ 2: `web`-এর ভেতরে `docker exec web wget -qO- http://api:3000/health` চালান — নামে ডাকা যায় কি না দেখুন।

ধাপ 3: `docker inspect api` দিয়ে api-র IP নিয়ে সেই IP-তে request পাঠান — IP-তেও কাজ করে, কিন্তু IP বদলায়, নাম বদলায় না।

ধাপ 4: ডিফল্ট bridge-এ `outsider` container চালিয়ে সেখান থেকে `http://api:3000/health` চেষ্টা করুন — ব্যর্থ হওয়ার কথা; ব্যর্থতাটিই প্রমাণ যে network বিচ্ছিন্নতা কাজ করছে।

ধাপ 5: `outsider`-কে `docker network connect technova-net outsider` দিয়ে যুক্ত করে আবার চেষ্টা করুন — এবার সাড়া আসবে।

ধাপ 6: Host থেকে `curl localhost:3000/health` চেষ্টা করুন — port publish না করায় ব্যর্থ হবে; তারপর বুঝুন কেন Compose-এ শুধু nginx-এর port বাইরে খোলা হয়।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker exec web wget -qO- http://api:3000/health
docker exec outsider wget -qO- --timeout=3 http://api:3000/health || echo "isolated: OK"
# প্রত্যাশিত: web থেকে ok, outsider থেকে ব্যর্থ — এটাই সঠিক ফল।
```

প্রত্যাশিত ফলাফল: একই network-এর container নামে একে অপরকে ডাকতে পারে, IP-তেও পারে; ভিন্ন network-এর container পৌঁছাতে পারে না; `connect` করলে যোগাযোগ খুলে যায়। Port publish না করলে হোস্ট থেকেও পৌঁছানো যায় না — internal যোগাযোগ ও বাইরের access আলাদা জিনিস।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
নামে ডাকা যাচ্ছে না, IP-তে যাচ্ছে	Container ডিফল্ট bridge-এ আছে — custom network-এ DNS কাজ করে, ডিফল্ট bridge-এ নামে ডাকা যায় না (ল্যাব ১৫)।
wget নেই বলছে	alpine-এ `wget` থাকে, কিন্তু কোনো image-এ না থাকলে `sh -c 'cat < /dev/tcp/api/3000'` বা `curl` দিয়ে পরীক্ষা করুন।
outsider থেকেও সাড়া আসছে	outsider ভুলে technova-net-এ চালু হয়েছে কি না `docker inspect outsider` দিয়ে দেখুন।
api container Up, তবু connection refused	অ্যাপ `0.0.0.0`-এ শুনছে কি না দেখুন (ল্যাব ৮) — শুধু localhost-এ শুনলে network-এর অন্য container পৌঁছাতে পারে না।

Troubleshooting

যোগাযোগ ব্যর্থ হলে ধাপে ধাপে যান: (১) দুটি container একই network-এ কি না (`docker inspect`), (২) লক্ষ্য container চলছে কি না (`docker ps`), (৩) অ্যাপ ভেতরে শুনছে কি না (`docker exec api wget -qO- http://localhost:3000/health` — নিজের ভেতরে কাজ করলে অ্যাপ ঠিক আছে, সমস্যা network-এ)।

Cleanup

`docker stop api web outsider && docker rm api web outsider; docker network rm technova-net`।

Reflection প্রশ্ন

১. Service-কে নামে ডাকা কেন IP-তে ডাকার চেয়ে ভালো — container restart-এ কী বদলায়? ২. Port publish না করেও container-দের মধ্যে যোগাযোগ হয় কীভাবে? ৩. Microservice-এ একটি সার্ভিসকে শুধু internal রাখা কেন নিরাপত্তার জন্য ভালো?

ল্যাব 15

Container DNS ও Service Discovery পরীক্ষা করা

উদ্দেশ্য (Objective): Docker-এর embedded DNS সার্ভার (127.0.0.11) কীভাবে container-এর নামকে IP-তে বদলায় তা পরীক্ষা করা — `nslookup`, `getent hosts`, alias ও `--link`-এর পুরনো পদ্ধতির সঙ্গে তুলনা। এই ল্যাবের পর service discovery-র রহস্য আর রহস্য থাকবে না।

পূর্বশর্ত (Prerequisites): ল্যাব ১৪ সম্পূর্ণ।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, technova-net, alpine ও nginx image।

আর্কিটেকচার:
Container → DNS query (নাম: api) → Docker embedded DNS 127.0.0.11 → technova-net-এর api container-এর বর্তমান IP → উত্তর → HTTP request সেই IP-তে যায় (container restart-এ IP বদলালেও নাম একই থাকে)

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# network ও container বানান
docker network create technova-net
docker run -d --name api --network technova-net technova-api:1.0
docker run -d --name checker --network technova-net alpine:3.20 sleep 3600

# container-এর ভেতরে DNS দেখুন
docker exec checker cat /etc/resolv.conf
# নমুনা আউটপুট — চিত্রণমূলক: nameserver 127.0.0.11

docker exec checker getent hosts api
docker exec checker getent hosts api technova-net
# নমুনা আউটপুট — চিত্রণমূলক: 172.25.0.2  api

# alias দিয়ে অতিরিক্ত নাম দিন
docker run -d --name api2 --network technova-net --network-alias backend --network-alias api-
v2 technova-api:1.0
docker exec checker getent hosts backend
docker exec checker getent hosts api-v2

# container restart করে IP বদলালেও নাম একই থাকে কি না দেখুন
docker restart api
docker exec checker getent hosts api
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `technova-net`-এ `api` ও `checker` (alpine, `sleep 3600` দিয়ে চালু রাখা) container চালান।

ধাপ 2: `checker`-এর ভেতরে `cat /etc/resolv.conf` দিয়ে দেখুন nameserver `127.0.0.11` — এটাই Docker-এর embedded DNS।

ধাপ 3: `getent hosts api` চালিয়ে নামের IP দেখুন, তারপর `docker inspect api`-এর IP-র সঙ্গে মিলিয়ে দেখুন।

ধাপ 4: `--network-alias backend` দিয়ে আরেকটি container চালিয়ে `getent hosts backend` পরীক্ষা করুন — একই container-এর একাধিক নাম থাকতে পারে।

ধাপ 5: `api` container restart করে আবার `getent hosts api` চালান — IP বদলালেও নাম একই IP-তে (নতুন IP-তে) মেলে কি না দেখুন।

ধাপ 6: ডিফল্ট bridge-এ একই পরীক্ষা করুন — `getent hosts`-এ নাম মেলে না, এটাই custom network-এর সুবিধা।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker exec checker cat /etc/resolv.conf
docker exec checker getent hosts api
docker exec checker getent hosts backend
# প্রত্যাশিত: nameserver 127.0.0.11; api ও backend — দুটো নামই IP দেয়।
```

প্রত্যাশিত ফলাফল: Embedded DNS (`127.0.0.11`) container-এর নামকে বর্তমান IP-তে বদলায়; alias-ও কাজ করে; restart-এ IP বদলালেও নামের উত্তর আপডেট হয়; ডিফল্ট bridge-এ নামে ডাকা যায় না।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
getent hosts-এ নাম পাওয়া যাচ্ছে না	দুটি container একই custom network-এ আছে কি না দেখুন; `--link` ব্যবহার করবেন না — সেটি পুরনো পদ্ধতি, custom network-ই আধুনিক উপায়।
resolv.conf-এ 127.0.0.11 নেই	Container ডিফল্ট bridge-এ চলছে — custom network-এ চালান।
nslookup নেই বলছে	alpine-এ `nslookup` থাকে না — `getent hosts` বা `ping` ব্যবহার করুন, অথবা `apk add bind-tools` দিয়ে ইনস্টল করুন।
alias কাজ করছে না	alias `--network technova-net --network-alias backend` — দুটি flag একসাথে দিতে হয়; শুধু `--network-alias` দিলে ডিফল্ট network-এ alias যায়।

Troubleshooting

DNS সমস্যা হলে প্রথমে `cat /etc/resolv.conf` দেখুন — nameserver 127.0.0.11 না থাকলে container custom network-এ নেই। তারপর `docker network inspect technova-net`-এ container-টি তালিকায় আছে কি না দেখুন। বাইরের ইন্টারনেটের DNS (যেমন google.com) না মিললে সমস্যা হোস্টের DNS-এ, Docker-এর embedded DNS-এ নয়।

Cleanup

`docker stop api api2 checker && docker rm api api2 checker; docker network rm technova-net`।

Reflection প্রশ্ন

১. Embedded DNS না থাকলে microservice-কে কীভাবে একে অপরকে খুঁজতে হতো (IP hardcode, বাইরের DNS)? ২. Compose-এ service-এর নামই DNS নাম হয় কেন — ল্যাব ১৮-এর সঙ্গে মিলিয়ে ভাবুন। ৩. Alias কখন কাজে লাগে — যেমন ডেটাবেসের নাম বদলালে পুরনো নামটিও চালু রাখতে?

ল্যাব 16

Volume ও Bind Mount তৈরি করা

উদ্দেশ্য (Objective): তিন ধরনের storage — volume, bind mount ও tmpfs mount — তৈরি করে পার্থক্য দেখা: volume Docker পরিচালনা করে, bind mount হোস্টের ফোল্ডার সরাসরি দেখায়, tmpfs RAM-এ থাকে। এই ল্যাবের পর আপনি জানবেন কখন কোনটি ব্যবহার করতে হয়।

পূর্বশর্ত (Prerequisites): ল্যাব ৫ সম্পূর্ণ; container মুছলে তার ভেতরের ফাইলও যায় — এই ধারণা আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, nginx image, হোস্টে `~/technova-data/` ফোল্ডার।

আর্কিটেকচার:
Container (লেখার layer — container মুছলে যায়) | Volume technova-data (/var/lib/docker/volumes/...) — container মুছলেও থাকে | Bind mount ~/technova-data → /data (হোস্টের ফাইল সরাসরি) | tmpfs /tmp/cache (RAM, container থামলে যায়)

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# 1) Named volume বানান ও ব্যবহার করুন
docker volume create technova-data
docker run -d --name vol-demo -v technova-data:/data nginx:1.27-alpine
docker exec vol-demo sh -c 'echo "hello volume" > /data/note.txt'
docker exec vol-demo cat /data/note.txt

# 2) Bind mount: হোস্টের ফোল্ডার container-এ দেখান
mkdir -p ~/technova-data
echo "hello host" > ~/technova-data/host-note.txt
docker run -d --name bind-demo -v ~/technova-data:/data nginx:1.27-alpine
docker exec bind-demo cat /data/host-note.txt
# হোস্ট থেকে ফাইল বদলান, container-এ সঙ্গে সঙ্গে বদলায়:
echo "changed on host" > ~/technova-data/host-note.txt
docker exec bind-demo cat /data/host-note.txt

# 3) tmpfs mount (RAM-এ, স্থায়ী নয়)
docker run -d --name tmp-demo --tmpfs /tmp/cache nginx:1.27-alpine

# volume-গুলো দেখুন
docker volume ls
docker volume inspect technova-data --format 'Mountpoint: {{.Mountpoint}}'
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `docker volume create technova-data` দিয়ে named volume বানান, তারপর `-v technova-data:/data` দিয়ে nginx container চালান।

ধাপ 2: Container-এর ভেতরে `/data/note.txt` লিখে `cat` দিয়ে দেখুন — ফাইলটি আছে।

ধাপ 3: Container মুছে (`docker rm -f vol-demo`) আবার একই volume দিয়ে নতুন container চালিয়ে দেখুন `note.txt` এখনো আছে — এটাই persistence।

ধাপ 4: হোস্টে `~/technova-data/host-note.txt` বানিয়ে bind mount দিয়ে container চালান — হোস্টের ফাইল container-এ দেখা যায়, হোস্টে বদলালে container-এও বদলায়।

ধাপ 5: `--tmpfs /tmp/cache` দিয়ে container চালিয়ে ভেতরে ফাইল লিখুন, container restart করে দেখুন ফাইলটি গেছে — tmpfs স্থায়ী নয়।

ধাপ 6: `docker volume inspect technova-data` দিয়ে হোস্টে volume-টি কোথায় আছে (Mountpoint) দেখুন — সরাসরি সেখানে ফাইল বদলাবেন না, Docker-এর মাধ্যমে ব্যবহার করুন।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker exec vol-demo cat /data/note.txt
docker volume ls
docker inspect vol-demo --format '{{range .Mounts}}{{.Type}}: {{.Source}} -> {{.Destination}}
{{end}}'
# প্রত্যাশিত: note.txt আছে; Mounts-এ type volume দেখা যায়।
```

প্রত্যাশিত ফলাফল: Named volume-এর ফাইল container মুছলেও থাকে; bind mount-এ হোস্টের ফাইল সরাসরি দেখা যায় ও দুদিকেই বদলায়; tmpfs restart-এ মুছে যায়; `docker volume inspect`-এ Mountpoint দেখা যায়।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
Bind mount-এ `permission denied`	হোস্টের ফোল্ডারের মালিকানা ও container-এর user মিলছে না — হোস্টে `chmod` বা container-এ non-root user-এর UID মেলান (ল্যাব ২১)।
Volume-এ ফাইল লিখেও container মুছলে ফাইল গেল	আপনি anonymous volume বা container-এর লেখার layer ব্যবহার করেছেন — named volume (`-v technova-data:/data`) ব্যবহার করুন, নাম ছাড়া `-v /data` নয়।
Bind mount-এ হোস্টের ফোল্ডার খালি দেখাচ্ছে	পথটি absolute কি না দেখুন — `-v ~/technova-data:/data`-এ `~` সবসময় expand হয় না; `$HOME/technova-data` বা পুরো পথ ব্যবহার করুন।
Volume মুছতে গিয়ে `volume is in use`	Volume ব্যবহার করা container এখনো আছে — আগে container মুছুন, তারপর `docker volume rm` দিন।

Troubleshooting

ফাইল দেখা না গেলে `docker inspect <name>`-এর `Mounts` অংশ দেখুন — Type (volume/bind/tmpfs), Source ও Destination ঠিক আছে কি না। Bind mount-এ হোস্টের পথ ভুল হলে Docker খালি ফোল্ডার বানিয়ে দেয় — তখন ফাইল “হারিয়ে” যায়নি, ভুল জায়গায় mount হয়েছে।

Cleanup

> **সতর্কতা — volume মোছার আগে:** `docker volume rm technova-data` চালালে ভেতরের সব ডেটা চিরতরে যাবে, ফিরিয়ে আনা যাবে না। এই ল্যাবে volume-টি রেখে দিন — ল্যাব ১৭-এ এটিই ব্যবহার হবে। শুধু `bind-demo` ও `tmp-demo` container মুছুন।

Reflection প্রশ্ন

১. ডেটাবেসের ডেটার জন্য volume কেন, bind mount কেন নয় — backup ও portability-র দিক থেকে ভাবুন? ২. Bind mount কখন ভালো — যেমন কোড বদলে সঙ্গে সঙ্গে container-এ দেখতে (development)? ৩. tmpfs কখন ব্যবহার করবেন — secret বা cache-এর জন্য কেন RAM ভালো?

ল্যাব 17

ডেটাবেস Persistence ও Recovery পরীক্ষা করা

উদ্দেশ্য (Objective): MongoDB container-এ volume দিয়ে ডেটা লিখে container মুছে আবার চালিয়ে ডেটা ফিরে পাওয়া, আর volume ছাড়া ডেটা হারিয়ে যাওয়া — দুটোই পরীক্ষা করা। এই ল্যাবটি প্রোডাকশন ডেটার সবচেয়ে গুরুত্বপূর্ণ পাঠ: container অস্থায়ী, volume স্থায়ী।

পূর্বশর্ত (Prerequisites): ল্যাব ১৬ সম্পূর্ণ; MongoDB-এর ধারণা আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, mongo:7 image, `mongosh` (container-এর ভেতরে আছে)।

আর্কিটেকচার:
MongoDB container + volume mongo-data (/data/db) → ডেটা লেখা → container rm → নতুন container + একই volume → ডেটা ফিরে আসে ✅ | Volume ছাড়া container → rm → ডেটা চিরতরে যায় ❌

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# volume দিয়ে MongoDB চালান
docker volume create mongo-data
docker run -d --name technova-mongo --network technova-net \
  -v mongo-data:/data/db \
  -e MONGO_INITDB_ROOT_USERNAME=admin \
  -e MONGO_INITDB_ROOT_PASSWORD=CHANGE-ME-MONGO-PASSWORD \
  mongo:7

# ডেটা লিখুন (mongosh container-এর ভেতরেই আছে)
docker exec technova-mongo mongosh -u admin -p CHANGE-ME-MONGO-PASSWORD --eval
'db.getSiblingDB("technova").users.insertOne({name:"Nahid", role:"developer"})'
docker exec technova-mongo mongosh -u admin -p CHANGE-ME-MONGO-PASSWORD --eval
'db.getSiblingDB("technova").users.countDocuments()'
# নমুনা আউটপুট — চিত্রণমূলক: 1

# container মুছে একই volume দিয়ে আবার চালান
docker stop technova-mongo && docker rm technova-mongo
docker run -d --name technova-mongo --network technova-net \
  -v mongo-data:/data/db \
  -e MONGO_INITDB_ROOT_USERNAME=admin \
  -e MONGO_INITDB_ROOT_PASSWORD=CHANGE-ME-MONGO-PASSWORD \
  mongo:7
sleep 5
docker exec technova-mongo mongosh -u admin -p CHANGE-ME-MONGO-PASSWORD --eval
'db.getSiblingDB("technova").users.find().toArray()'
# নমুনা আউটপুট — চিত্রণমূলক: [{ name: "Nahid", role: "developer" }] — ডেটা ফিরে এসেছে!

# তুলনা: volume ছাড়া MongoDB (পরীক্ষার জন্য আলাদা নামে)
docker run -d --name mongo-novolume mongo:7
# ... ডেটা লিখুন, container মুছুন — ডেটা আর ফিরবে না (পরীক্ষা করে দেখুন)
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `docker volume create mongo-data` বানান, তারপর উপরের কমান্ডে MongoDB চালান — পাসওয়ার্ডটি কাল্পনিক, আসল পাসওয়ার্ড এখানে ব্যবহার করবেন না।

ধাপ 2: `mongosh` দিয়ে `technova` ডেটাবেসে একটি user লিখুন ও `countDocuments()` দিয়ে 1 দেখুন।

ধাপ 3: Container stop করে rm করুন, তারপর একই volume দিয়ে আবার চালান — 5 সেকেন্ড অপেক্ষা করে `find()` দিয়ে দেখুন ডেটা ফিরে এসেছে।

ধাপ 4: Volume ছাড়া আরেকটি MongoDB (`mongo-novolume`) চালিয়ে একইভাবে ডেটা লিখুন, container মুছুন, আবার চালান — ডেটা নেই, এটাই প্রমাণ।

ধাপ 5: `docker volume inspect mongo-data` দিয়ে Mountpoint দেখুন; `docker exec technova-mongo ls /data/db` দিয়ে ডেটা ফাইল দেখুন।

ধাপ 6: Backup পরীক্ষা করুন: `docker exec technova-mongo mongodump -u admin -p CHANGE-ME-MONGO-PASSWORD --out /tmp/backup` চালিয়ে backup নেওয়ার ধারণা নিন (পূর্ণ backup প্রজেক্ট ৩-এ)।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker exec technova-mongo mongosh -u admin -p CHANGE-ME-MONGO-PASSWORD --eval
'db.getSiblingDB("technova").users.countDocuments()'
# প্রত্যাশিত: 1 — container মুছে আবার চালানোর পরেও ডেটা আছে।
```

প্রত্যাশিত ফলাফল: Volume-সহ MongoDB-র ডেটা container মুছে আবার চালানোর পরেও ফিরে আসে; volume ছাড়া ডেটা হারিয়ে যায়; `mongosh`-এ count 1 দেখা যায়। এই পার্থক্যই প্রোডাকশনে volume বাধ্যতামূলক করে।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
mongosh-এ `Authentication failed`	Username/password ঠিক আছে কি না দেখুন — প্রথম চালুতে দেওয়া credential-ই স্থায়ী হয়; volume-এ পুরনো credential থাকলে নতুন password কাজ করবে না, volume মুছে আবার শুরু করতে হয়।
MongoDB চালু হতে সময় লাগছে	`docker logs technova-mongo` দেখুন — “waiting for connections” না আসা পর্যন্ত mongosh চালাবেন না; ল্যাব ২০-এর health check এই অপেক্ষাটি স্বয়ংক্রিয় করে।
Volume-এ ডেটা থাকা সত্ত্বেও ডেটা দেখাচ্ছে না	ভুল volume mount করেছেন কি না দেখুন — `docker inspect technova-mongo`-এর Mounts-এ Source `mongo-data` কি না মিলিয়ে দেখুন।
mongodump-এ permission denied	Backup ফোল্ডারটি volume-এ বা bind mount-এ লিখুন, container-এর read-only জায়গায় নয়।

Troubleshooting

ডেটা না মিললে প্রথমে `docker logs technova-mongo` দেখুন MongoDB চালু হয়েছে কি না, তারপর `docker volume ls` ও `docker inspect` দিয়ে সঠিক volume mount হয়েছে কি না দেখুন। ভুল password-এ আটকে গেলে volume-টি মুছে (ডেটা যাবে — ল্যাবের ডেটা বলে সমস্যা নেই) আবার শুরু করুন; প্রোডাকশনে এটা করবেন না, সেখানে password বদলানোর আলাদা পদ্ধতি আছে।

Cleanup

> **সতর্কতা — ডেটা সুরক্ষা:** এই ল্যাবের `mongo-data` volume মুছবেন না — ল্যাব ১৮-এর Compose পরিবেশে এটিই ব্যবহার হবে। `mongo-novolume` container মুছে ফেলুন। `docker volume rm mongo-data` কখনো ভুলেও চালাবেন না যতক্ষণ না backup নেওয়া আছে।

Reflection প্রশ্ন

১. Container মুছলে ডেটা যায়, volume মুছলে ডেটা যায় — দুটোর মধ্যে পার্থক্য কী, আর কেন volume-ই ডেটার ঠিকানা? ২. প্রথম চালুতে দেওয়া MongoDB password কেন পরে বদলানো যায় না (volume-এ কী জমে থাকে)? ৩. প্রোডাকশনে শুধু volume-ই কি যথেষ্ট, নাকি backup-ও লাগে — volume-এর disk নষ্ট হলে কী হবে?

ল্যাব 18

Docker Compose দিয়ে MERN পরিবেশ তৈরি করা

উদ্দেশ্য (Objective): একটি `compose.yaml` ফাইলে frontend, backend, MongoDB ও Nginx — চারটি সার্ভিস একসাথে সংজ্ঞায়িত করে `docker compose up` দিয়ে পুরো MERN পরিবেশ চালানো। এই ল্যাবটি বইয়ের কেন্দ্রীয় দক্ষতা: একটি ফাইলে পুরো অ্যাপ।

পূর্বশর্ত (Prerequisites): ল্যাব ৮, ১৩–১৭ সম্পূর্ণ; technova-api কোড আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine + Compose v2, technova-api প্রজেক্ট, `~/technova-app/` ফোল্ডার।

আর্কিটেকচার:

docker compose up → technova-net (custom network) → mongo (volume mongo-data) ← api (technova-api:3000) ← nginx (80 → api) ← Host :8080 → ব্রাউজার | client (React) → nginx → api → mongo

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# ~/technova-app/compose.yaml (version: key লিখবেন না — Compose v2-এ দরকার নেই)
services:
  mongo:
    image: mongo:7
    container_name: technova-mongo
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: CHANGE-ME-MONGO-PASSWORD
    volumes:
      - mongo-data:/data/db
    networks: [technova-net]
  api:
    build: ./server
    container_name: technova-api
    restart: unless-stopped
    environment:
      PORT: 3000
      MONGO_URI: mongodb://admin:CHANGE-ME-MONGO-PASSWORD@mongo:27017/technova?
authSource=admin
    depends_on: [mongo]
    networks: [technova-net]
  nginx:
    image: nginx:1.27-alpine
    container_name: technova-nginx
    restart: unless-stopped
    ports: ["8080:80"]
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on: [api]
    networks: [technova-net]
volumes:
  mongo-data:
```

```
networks:
  technova-net:
    driver: bridge

# ~/technova-app/nginx/default.conf
# server { listen 80; location / { proxy_pass http://api:3000; proxy_set_header Host
$host; } }

# চালান
docker compose -f ~/technova-app/compose.yaml up -d --build
docker compose -f ~/technova-app/compose.yaml ps
curl -s http://localhost:8080/health
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `~/technova-app/`-এ উপরের `compose.yaml` ও `nginx/default.conf` লিখুন — লক্ষ্য করুন `api` সার্ভিসটি `mongo` নামে ডাকছে (DNS, ল্যাব ১৫), আর nginx `api:3000`-এ proxy করছে।

ধাপ 2: `docker compose up -d --build` চালান — `--build` api image-টি নতুন করে বানায়; log-এ প্রতিটি সার্ভিসের image pull/build দেখুন।

ধাপ 3: `docker compose ps` দিয়ে চারটি সার্ভিসের অবস্থা দেখুন — সবগুলো Up কি না।

ধাপ 4: `curl http://localhost:8080/health` দিয়ে nginx → api → (mongo) পথটি পরীক্ষা করুন — শুধু nginx-এর port-ই হোস্টে খোলা, api ও mongo internal।

ধাপ 5: `docker compose logs api` ও `docker compose logs mongo` দিয়ে প্রতিটি সার্ভিসের log আলাদা করে দেখুন।

ধাপ 6: একটি সার্ভিস restart করুন: `docker compose restart api` — অন্য সার্ভিসগুলো চলতে থাকে, এটাই Compose-এর সুবিধা।

ধাপ 7: `docker compose down` দিয়ে থামান, তারপর `docker compose up -d` দিয়ে আবার চালান — volume থাকায় MongoDB-র ডেটা ফিরে আসে কি না দেখুন।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker compose ps --format 'table {{.Name}}\t{{.Status}}\t{{.Ports}}'
curl -s http://localhost:8080/health
# প্রত্যাশিত: তিনটি সার্ভিস Up; /health-এ {"status":"ok"} আসে।
# নমুনা আউটপুট — চিত্রণমূলক: technova-nginx Up 0.0.0.0:8080->80/tcp
```

প্রত্যাশিত ফলাফল: একটি `compose.yaml` থেকে তিনটি সার্ভিস একসাথে চলে, nginx-এর মাধ্যমে API পৌঁছায়, `docker compose ps`-এ সব Up, `down` করে আবার `up` করলেও MongoDB-র ডেটা থাকে।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান

`docker compose` কমান্ড পাওয়া যাচ্ছে না	Compose plugin ইনস্টল হয়নি — ল্যাব ৩-এর `docker-compose-plugin` ইনস্টল করুন; `docker-compose` (hyphen) পুরনো কমান্ড, ব্যবহার করবেন না।
api সার্ভিসে `failed to resolve mongo`	`api` ও `mongo` একই network-এ আছে কি না দেখুন — Compose-এ সব সার্ভিস ডিফল্টভাবে একই network-এ থাকে, তবে `networks:` আলাদা করে দিলে মিলিয়ে দিতে হয়।
nginx-এ `host not found in upstream api`	api সার্ভিস চালু হওয়ার আগেই nginx config load করেছে — `depends_on` শুধু চালুর ক্রম দেয়, প্রস্তুতি নয়; ল্যাব ২০-এর health check এটাই ঠিক করে।
compose.yaml-এ `version is obsolete` সতর্কতা	`version:` লাইনটি মুছে ফেলুন — Compose v2-এ এটি অপ্রয়োজনীয় ও সতর্কতা দেয়।

Troubleshooting

কোনো সার্ভিস না চললে `docker compose ps` দিয়ে কোনটি Exited তা দেখুন, তারপর `docker compose logs <service>` দিয়ে তার log দেখুন — প্রায় সবসময় কারণ সেখানেই থাকে। `docker compose config` দিয়ে Compose ফাইলটি যাচাই করুন — YAML ভুল থাকলে সেখানেই ধরা পড়ে, চালানোর আগেই।

Cleanup

`docker compose down` দিয়ে থামান — volume মুছবেন না (`down -v` দেবেন না, তাতে mongo-data মুছে যাবে)। Compose ফাইলটি রেখে দিন, পরের ল্যাবগুলোতে এটিই বাড়ানো হবে।

Reflection প্রশ্ন

১. আলাদা আলাদা `docker run`-এর বদলে Compose ব্যবহারের সুবিধা কী — একটি ফাইলে কী কী বাঁধা থাকে? ২. `depends_on` কী গ্যারান্টি দেয়, আর কী দেয় না? ৩. শুধু nginx-এর port বাইরে খোলা কেন — api ও mongo-র port বাইরে খুললে কী ঝুঁকি?

ল্যাব 19

Environment Variable কনফিগার করা

উদ্দেশ্য (Objective): Environment variable দিয়ে একই image-কে dev, staging ও production-এ আলাদা আচরণ করানো — `.env` ফাইল, Compose-এর `environment:` ও `env_file:`, আর secret image-এ না রাখার নিয়ম। এই ল্যাবের পর configuration কোড থেকে আলাদা থাকবে।

পূর্বশর্ত (Prerequisites): ল্যাব ১৮ সম্পূর্ণ; compose.yaml আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine + Compose v2, technova-app প্রজেক্ট।

আর্কিটেকচার:
.env ফাইল (হোস্টে, git-এ যাবে না) → docker compose (env_file / environment) → Container-এর env → অ্যাপ process.env পড়ে → একই image, ভিন্ন পরিবেশে ভিন্ন আচরণ

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# ~/technova-app/.env (কাল্পনিক মান — আসল secret এখানে লিখবেন না)
MONGO_USER=admin
MONGO_PASSWORD=CHANGE-ME-MONGO-PASSWORD
MONGO_DB=technova
NODE_ENV=development
PORT=3000
API_MESSAGE=Hello from TechNova dev

# compose.yaml-এ সার্ভিসের environment অংশ (আংশিক)
#   api:
#     env_file: [.env]
#     environment:
#       NODE_ENV: ${NODE_ENV:-development}
#       MONGO_URI: mongodb://${MONGO_USER}:${MONGO_PASSWORD}@mongo:27017/${MONGO_DB}?
authSource=admin
#   mongo:
#     environment:
#       MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER}
#       MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}

# container-এর ভেতরে env দেখুন (secret-এর মান log-এ ছাপাবেন না)
docker compose exec api sh -c 'echo $NODE_ENV; echo $PORT'
docker compose exec api printenv | grep -v PASSWORD | grep -v SECRET

# .env ছাড়া একবার চালিয়ে দেখুন কী হয়
# docker compose --env-file /dev/null up -d  (পরীক্ষার জন্য — তারপর আগের মতো ফিরিয়ে আনুন)
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `~/technova-app/.env` ফাইল লিখুন — কাল্পনিক মান দিয়ে; তারপর `.gitignore`-এ `.env` যোগ করুন ও `.dockerignore`-এও `.env` আছে কি না দেখুন।

ধাপ 2: compose.yaml-এ `api` সার্ভিসে `env_file: [.env]` ও `environment:`-এ `${MONGO_USER}` ধরনের variable substitution যোগ করুন।

ধাপ 3: `docker compose up -d` চালিয়ে `docker compose exec api printenv | grep -v PASSWORD` দিয়ে দেখুন variable-গুলো container-এ পৌঁছেছে কি না — password-এর মান কখনো print করবেন না।

ধাপ 4: `.env`-এ `API_MESSAGE` বদলে `docker compose up -d` দিয়ে আবার চালান — অ্যাপের আচরণ বদলায় কি না দেখুন (অ্যাপকে env পড়তে হয় — server.js-এ `process.env.API_MESSAGE` ব্যবহার করুন)।

ধাপ 5: একটি variable `docker compose exec -e FOO=bar api ...` দিয়ে একবারের জন্য override করে দেখুন।

ধাপ 6: `docker compose config` দিয়ে দেখুন Compose variable-গুলো কীভাবে resolve করেছে — password-এর মান সেখানে দেখা গেলে সেটি শেয়ার করবেন না, শুধু নিজে যাচাই করুন।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker compose exec api sh -c 'echo NODE_ENV=$NODE_ENV PORT=$PORT'
docker compose config | grep -c 'CHANGE-ME' || true
# প্রত্যাশিত: NODE_ENV=development PORT=3000 দেখা যায়;
# config-এ variable resolve হয়েছে।
```

প্রত্যাশিত ফলাফল: একই image `.env` বদলে ভিন্ন আচরণ করে; variable-গুলো container-এ পৌঁছায়; `.env` git-এ যায় না; `docker compose config`-এ resolve করা মান দেখা যায়।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
Variable খালি আসছে	`.env` ফাইলটি compose.yaml-এর পাশে আছে কি না দেখুন; `env_file`-এর পথ compose ফাইলের সাপেক্ষে হয়।
Password বদলেও MongoDB পুরনো password চাইছে	ল্যাব ১৭-এর মতো — প্রথম চালুতে তৈরি হওয়া credential volume-এ জমে থাকে; নতুন password-এ নতুন volume লাগে (ল্যাবের ক্ষেত্রে volume মুছে আবার শুরু করুন)।
Secret image-এ ঢুকে গেছে	`ENV` দিয়ে Dockerfile-এ secret লিখেছেন কি না দেখুন — কখনো লিখবেন না; `docker history`-এ secret দেখা গেলে image-টি মুছে ফেলুন ও Hub থেকে সরান।
`${VAR}` resolve হচ্ছে না	Compose-এ `$` escape করতে `$${VAR}` লিখতে হয় — না হলে Compose নিজে resolve করার চেষ্টা করে।

Troubleshooting

Env সমস্যা হলে `docker compose config` দিয়ে প্রথমে দেখুন Compose কী resolve করেছে — সমস্যা Compose-এ না অ্যাপে, তা এখানেই আলাদা হয়। তারপর container-এর ভেতরে `printenv` (password বাদ দিয়ে) দিয়ে দেখুন variable পৌঁছেছে কি না। Secret-এর মান কখনো log, স্ক্রিনশট বা চ্যাটে দেবেন না।

Cleanup

`docker compose down` দিয়ে থামান; `.env` ফাইলটি রেখে দিন (ল্যাবের কাল্পনিক মান বলে সমস্যা নেই, তবে আসল secret থাকলে মুছে ফেলুন)।

Reflection প্রশ্ন

১. Configuration কোড থেকে আলাদা রাখা কেন ভালো — একই image তিন পরিবেশে চলার সুবিধা কী? ২. `environment:` বনাম `env_file:` — কখন কোনটি? ৩. Secret-এর জন্য `.env`-ই কি যথেষ্ট, নাকি প্রোডাকশনে Docker secret / vault লাগে?

ল্যাব 20

Health Check ও Service Dependency বাস্তবায়ন করা

উদ্দেশ্য (Objective): Container-এর health check (`HEALTHCHECK` / Compose `healthcheck:`) ও `depends_on: condition: service_healthy` দিয়ে সার্ভিসগুলোকে প্রস্তুতির ক্রমে চালানো। এই ল্যাবের পর nginx আর api প্রস্তুত হওয়ার আগেই request পাঠাবে না, api-ও MongoDB প্রস্তুত হওয়ার আগে সংযোগ চাইবে না।

পূর্বশর্ত (Prerequisites): ল্যাব ১৮–১৯ সম্পূর্ণ; compose.yaml আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine + Compose v2, technova-app প্রজেক্ট, `wget` বা Node.js health endpoint।

আর্কিটেকচার:
mongo (healthcheck: mongosh ping) → healthy হলে api শুরু (healthcheck: wget /health) → healthy হলে nginx শুরু → Host :8080 ✅ | healthcheck ছাড়া: nginx আগেই শুরু হয়ে 'host not found' দেয় ❌

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# compose.yaml-এ healthcheck যোগ করুন (আংশিক)
#   mongo:
#     healthcheck:
#       test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')", "-u", "admin", "-p",
"CHANGE-ME-MONGO-PASSWORD"]
#       interval: 10s
#       timeout: 5s
#       retries: 5
#       start_period: 20s
#   api:
#     depends_on:
#       mongo: { condition: service_healthy }
#     healthcheck:
#       test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://localhost:3000/health"]
#       interval: 10s
#       timeout: 5s
#       retries: 5
#       start_period: 10s
#   nginx:
#     depends_on:
#       api: { condition: service_healthy }

# চালান ও health দেখুন
docker compose up -d --build
docker compose ps --format 'table {{.Name}}\t{{.Status}}'
# প্রত্যাশিত: (healthy) লেখা দেখা যাবে
docker inspect technova-api --format '{{.State.Health.Status}}'
# নমুনা আউটপুট — চিত্রণমূলক: healthy
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: compose.yaml-এ `mongo` সার্ভিসে `healthcheck` যোগ করুন — `mongosh --eval db.adminCommand('ping')` দিয়ে MongoDB সাড়া দিচ্ছে কি না পরীক্ষা হয়।

ধাপ 2: `api` সার্ভিসে `depends_on: mongo: condition: service_healthy` ও নিজের healthcheck (`wget /health`) যোগ করুন — alpine image-এ `wget` আছে, `curl` নেই, তাই `wget` ব্যবহার করুন।

ধাপ 3: `nginx`-এ `depends_on: api: condition: service_healthy` যোগ করুন।

ধাপ 4: `docker compose up -d --build` চালিয়ে `docker compose ps`-এ প্রতিটি সার্ভিসের পাশে `(healthy)` লেখা দেখুন — `(health: starting)` থেকে `(healthy)` হতে কত সময় লাগে তা লিখুন।

ধাপ 5: `docker inspect technova-api --format '{{.State.Health.Status}}'` ও `{{.State.Health.Log}}` দিয়ে শেষ health check-এর ফল দেখুন।

ধাপ 6: ইচ্ছা করে api-র `/health` ভাঙুন (server.js-এ ভুল করুন) — দেখুন api `(unhealthy)` হয়, আর nginx আর শুরু হয় না; তারপর ঠিক করে আবার চালান।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker compose ps
docker inspect technova-api --format 'Health: {{.State.Health.Status}}'
curl -s http://localhost:8080/health
# প্রত্যাশিত: সব সার্ভিস (healthy); /health-এ ok আসে।
```

প্রত্যাশিত ফলাফল: সব সার্ভিস `(healthy)` হয়; `depends_on: service_healthy`-এর কারণে সার্ভিসগুলো প্রস্তুতির ক্রমে চলে; ভাঙা `/health`-এ api `unhealthy` হয়।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
Health check-এ `wget: not found`	Base image-এ `wget` নেই — node:20-alpine-এ `wget` থাকে, কিন্তু অন্য image-এ না থাকলে Node.js-এর নিজের HTTP বা `node -e` দিয়ে health check লিখুন।
MongoDB health check সবসময় unhealthy	Username/password ভুল, অথবা `start_period` খুব ছোট — MongoDB চালু হতে 10–20 সেকেন্ড লাগে, `start_period: 20s` দিন।
`depends_on` দিয়েও api আগেই শুরু হচ্ছে	`depends_on: [mongo]` (তালিকা) শুধু ক্রম দেয়, প্রস্তুতি নয় — `condition: service_healthy` (mapping) ব্যবহার করতে হয়।
Health check-এ container ধীর হয়ে যাচ্ছে	`interval` খুব ছোট (যেমন 1s) দেবেন না — 10s যথেষ্ট; প্রতিটি check-ই একটি প্রসেস চালায়।

Troubleshooting

Health check ব্যর্থ হলে `docker inspect <container>`-এর `State.Health.Log` দেখুন — শেষ কয়েকটি check-এর output সেখানে থাকে। অ্যাপের log-ও দেখুন: health endpoint নিজে কাজ করছে কি না, তা `docker exec api wget -qO- http://localhost:3000/health` দিয়ে ভেতর থেকে পরীক্ষা করুন।

Cleanup

`docker compose down` দিয়ে থামান; healthcheck-সহ compose.yaml রেখে দিন — এটাই প্রজেক্ট ২-এর ভিত্তি।

Reflection প্রশ্ন

১. Health check বনাম readiness বনাম liveness — container চলছে মানেই কি সার্ভিস প্রস্তুত? ২. `depends_on: service_healthy` না থাকলে কী ভুল হয় — race condition কাকে বলে? ৩. Health check-এর `start_period` কেন দরকার — ধীরে চালু হওয়া ডেটাবেসের ক্ষেত্রে কী হয়?

ল্যাব 21

Non-root User দিয়ে Container চালানো

উদ্দেশ্য (Objective): Container-কে root-এর বদলে সাধারণ user (`node`) দিয়ে চালানো — Dockerfile-এ `USER` , `--chown` , ফাইলের মালিকানা ও port-এর সীমা। এই ল্যাবের পর আপনার container ভেঙে গেলেও আক্রমণকারী root পাবে না।

পূর্বশর্ত (Prerequisites): ল্যাব ৮ ও ১০ সম্পূর্ণ; technova-api Dockerfile আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, technova-api প্রজেক্ট।

আর্কিটেকচার:
Root container (UID 0 — হোস্টে root-এর সমান ক্ষমতা, ভাঙলে বিপদ) → Non-root container (USER node, UID 1000 — ফাইলে লেখা, port খোলা, package install — সব সীমিত) → ভাঙলেও ক্ষতি সীমিত

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# Dockerfile-এ non-root user যোগ করুন (আংশিক — ল্যাব ৮-এর Dockerfile-এর শেষে)
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --chown=node:node . ./
USER node
EXPOSE 3000
CMD ["node", "server.js"]

# build ও চালান
docker build -t technova-api:nonroot ./server
docker run -d --name api-nonroot -p 3003:3000 technova-api:nonroot
docker exec api-nonroot whoami
docker exec api-nonroot id
# নমুনা আউটপুট — চিত্রণমূলক: node, uid=1000(node) gid=1000(node)

# তুলনা: root-এ চলা container-এ whoami কী দেয় দেখুন
docker run -d --name api-root technova-api:1.0
docker exec api-root whoami
# নমুনা আউটপুট — চিত্রণমূলক: root

# non-root container-এ root-এর কাজ চেষ্টা করুন (ব্যর্থ হওয়ার কথা)
docker exec api-nonroot sh -c 'touch /etc/testfile' || echo "EXPECTED: permission denied"
curl -s http://localhost:3003/health
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: ল্যাব ৮-এর Dockerfile-এ `COPY --chown=node:node . ./` ও `USER node` যোগ করুন — `USER` লাইনটি `CMD` -এর আগে, `RUN npm ci` -এর পরে থাকতে হবে (install-এ root লাগে, চালাতে লাগে না)।

ধাপ 2: `docker build -t technova-api:nonroot ./server` চালান, তারপর চালিয়ে `docker exec api-nonroot whoami` ও `id` দিয়ে UID 1000 দেখুন।

ধাপ 3: `curl http://localhost:3003/health` দিয়ে যাচাই করুন non-root container-ও একই API দেয়।

ধাপ 4: Non-root container-এ `touch /etc/testfile` চেষ্টা করুন — permission denied হওয়ার কথা; এটাই নিরাপত্তা।

ধাপ 5: Root-এ চলা পুরনো image (`technova-api:1.0`) থেকে container চালিয়ে `whoami` তুলনা করুন — root বনাম node।

ধাপ 6: Volume mount করে দেখুন: non-root container volume-এ লিখতে পারে কি না — না পারলে হোস্টের ফোল্ডারের মালিকানা (`chown 1000:1000`) বা volume-এর permission ঠিক করুন।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker exec api-nonroot whoami
docker exec api-nonroot id
curl -s http://localhost:3003/health
# প্রত্যাশিত: whoami = node, uid=1000, /health-এ ok।
```

প্রত্যাশিত ফলাফল: Non-root image থেকে container `node` user-এ (UID 1000) চলে, একই API দেয়, root-এর জায়গায় লিখতে পারে না; root image-এর container `root` দেখায়।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
Build-এ `permission denied` — `USER node` খুব আগে দেওয়া	`USER` লাইনটি `RUN npm ci` -এর পরে দিন — install-এর সময় root লাগে; শুধু চালানোর সময় non-root দরকার।
Non-root container-এ `EACCES` ফাইল লিখতে গিয়ে	`COPY` করা ফাইল root-এর মালিকানায় আছে — `--chown=node:node` দিন, অথবা `RUN chown -R node:node /app` যোগ করুন।
Port 80-এ শুনতে পারছে না	1024-এর নিচের port-এ শুনতে root লাগে — অ্যাপকে 3000-এর মতো উঁচু port-এ শুনতে দিন, বাইরে nginx 80 থেকে proxy করবে।
Volume-এ লিখতে পারছে না	হোস্টের ফোল্ডারের মালিকানা root-এর — `sudo chown -R 1000:1000 ~/technova-data` দিয়ে মেলান, অথবা named volume ব্যবহার করুন (Docker নিজে মালিকানা সামলায়)।

Troubleshooting

Permission সমস্যা হলে `docker exec <name> id` দিয়ে container কোন UID-তে চলছে দেখুন, তারপর `ls -la /app` দিয়ে ফাইলের মালিকানা দেখুন — দুটো মিলছে কি না, সেটাই প্রশ্ন। `docker run --user 1000:1000` দিয়েও চালিয়ে দেখতে পারেন Dockerfile না বদলে — তবে Dockerfile-এ `USER` লেখাই স্থায়ী সমাধান।

Cleanup

`docker stop api-nonroot api-root && docker rm api-nonroot api-root; technova-api:nonroot` image রাখুন — প্রজেক্ট ১-এ এটিই ব্যবহার হবে।

Reflection প্রশ্ন

১. Container-এর root আর হোস্টের root কি এক? — user namespace ছাড়া কেন প্রায় এক? ২. `USER node` দিলেই কি নিরাপত্তা সম্পূর্ণ হয় — আর কী কী লাগে (read-only filesystem, capability drop)? ৩. সব অ্যাপ কি non-root-এ চলে — কোন অ্যাপে সমস্যা হতে পারে?

ল্যাব 22

Resource Limit প্রয়োগ ও Log পরিদর্শন করা

উদ্দেশ্য (Objective): Container-এ CPU ও মেমরি limit (`--memory`, `--cpus`) দেওয়া, limit ছাড়িয়ে গেলে OOMKilled হওয়া দেখা, আর `docker logs` দিয়ে log পড়া, ঘোরানো ও সীমিত করা। এই ল্যাবের পর একটি container আর পুরো হোস্টকে থামিয়ে দিতে পারবে না।

পূর্বশর্ত (Prerequisites): ল্যাব ৫ সম্পূর্ণ; `docker stats` দেখেছেন।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine, nginx ও technova-api image, `stress` পরীক্ষার জন্য (ঐচ্ছিক)।

আর্কিটেকচার:
Host (4 GB RAM) → Container A (limit 512 MB — OOM হলে শুধু A মরে) | Container B (limit 512 MB — চলতে থাকে) | Limit ছাড়া: একটি container সব RAM খেয়ে হোস্টকে থামিয়ে দেয় ❌

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# মেমরি limit দিয়ে container চালান
docker run -d --name limited-api --memory 512m --cpus 0.5 -p 3004:3000 technova-api:nonroot
docker stats --no-stream limited-api
docker inspect limited-api --format 'Memory: {{.HostConfig.Memory}} NanoCpus:
{{.HostConfig.NanoCpus}}'
# নমুনা আউটপুট — চিত্রণমূলক: Memory: 536870912 NanoCpus: 500000000

# limit ছাড়িয়ে গেলে কী হয়: মেমরি খাওয়া পরীক্ষা (সাবধানে — ল্যাব VM-এ করুন)
docker run -d --name mem-hog --memory 100m alpine:3.20 sh -c 'a=""; while true; do a="$a $a $a
$a"; done'
sleep 5
docker inspect mem-hog --format 'OOMKilled: {{.State.OOMKilled}} Status: {{.State.Status}}'
# নমুনা আউটপুট — চিত্রণমূলক: OOMKilled: true Status: exited

# log দেখা: follow, tail, timestamp
docker logs --tail 20 limited-api
docker logs -f --tail 5 limited-api &
sleep 3; curl -s http://localhost:3004/health > /dev/null; sleep 2; kill %1
docker logs --timestamps --tail 5 limited-api

# log-এর আকার সীমিত করা (Compose-এ)
#   logging:
#     driver: json-file
#     options: { max-size: "10m", max-file: "3" }
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `--memory 512m --cpus 0.5` দিয়ে api container চালিয়ে `docker stats`-এ LIMIT 512 MiB দেখুন — limit ছাড়া চালানো container-এর LIMIT হোস্টের মোট RAM দেখায়, তুলনা করুন।

ধাপ 2: `docker inspect --format '{{.HostConfig.Memory}}'` দিয়ে byte-এ limit যাচাই করুন (512m = 536870912 byte)।

ধাপ 3: `mem-hog` পরীক্ষাটি চালিয়ে 5 সেকেন্ড পর `docker inspect`-এ `OOMKilled: true` দেখুন — limit কাজ করছে, container মারা গেছে, হোস্ট বেঁচে আছে।

ধাপ 4: `docker logs --tail 20`, `docker logs -f` (follow) ও `--timestamps` দিয়ে log পড়ুন — request করলে log-এ কী যোগ হয় তা দেখুন।

ধাপ 5: `docker logs limited-api 2>&1 | grep -i error` দিয়ে error খুঁজুন — log-এ grep করা যায়, এটাই দৈনন্দিন কাজ।

ধাপ 6: Compose-এ `logging: options: {max-size: "10m", max-file: "3"}` যোগ করে দেখুন — log ফাইল আর অসীম বাড়ে না।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker stats --no-stream limited-api
docker inspect mem-hog --format 'OOMKilled: {{.State.OOMKilled}}'
docker logs --tail 5 limited-api
# প্রত্যাশিত: LIMIT 512MiB; OOMKilled true; log-এ listening লাইন।
```

প্রত্যাশিত ফলাফল: Limit-সহ container-এ `docker stats`-এ 512 MiB limit দেখা যায়; মেমরি খাওয়া container OOMKilled হয়ে মরে, হোস্ট বেঁচে থাকে; log follow, tail ও timestamp-সহ পড়া যায়।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
Limit দিয়েও container সব RAM খাচ্ছে	`--memory` শুধু RAM-এর limit; swap-ও সীমিত করতে `--memory-swap` দিন, না হলে swap ব্যবহার করে limit এড়ানো যায়।
OOMKilled না হয়ে container ধীর হয়ে যাচ্ছে	মেমরি limit-এর কাছাকাছি গেলে অ্যাপ GC (garbage collection) করে ধীর হয় — limit খুব ছোট দিলে এমন হয়; limit বাড়ান।
`docker logs` খালি	অ্যাপ stdout-এ লগ লিখছে না, ফাইলে লিখছে — stdout-এ লগ লেখা container-এর নিয়ম; ফাইলে লেখা log `docker logs`-এ আসে না।
Log ফাইল disk ভরে ফেলছে	`docker system df` দেখুন — json-file driver-এ `max-size` না দিলে log অসীম বাড়ে; Compose-এ limit যোগ করুন।

Troubleshooting

OOMKilled দেখলে `docker inspect`-এর `State.OOMKilled` ও `docker stats`-এর ইতিহাস দেখুন — অ্যাপের আসল মেমরি চাহিদা কত, তা মেপে limit ঠিক করুন। Log সমস্যা হলে `docker logs --tail 100` দিয়ে শুরু করুন, তারপর `--since` (যেমন `--since 10m`) দিয়ে সময়ের পরিসর ছোট করুন।

Cleanup

`docker stop limited-api mem-hog && docker rm limited-api mem-hog`। Log limit-সহ Compose কনফিগারেশন নোট করে রাখুন — প্রজেক্ট ৩-এ লাগবে।

Reflection প্রশ্ন

১. Limit না দিলে একটি container কীভাবে পুরো হোস্টকে থামিয়ে দিতে পারে — noisy neighbour কাকে বলে? ২. `--memory` বনাম `--memory-swap` — দুটোর পার্থক্য কী? ৩. Container-এর log কেন stdout-এ লেখা উচিত, ফাইলে নয়?

ল্যাব 23

Container আপডেট ও Rollback পরীক্ষা করা

উদ্দেশ্য (Objective): চলমান container-এর নতুন image সংস্করণে আপডেট করা (`v1 → v2`), আপডেট ব্যর্থ হলে আগের সংস্করণে rollback করা, আর `docker compose`-এ `pull` + `up` দিয়ে একই কাজ করা। এই ল্যাবের পর প্রোডাকশন আপডেট আর ভয়ের বিষয় থাকবে না।

পূর্বশর্ত (Prerequisites): ল্যাব ৮ ও ১৮ সম্পূর্ণ; technova-api কোড আছে।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine + Compose v2, technova-api প্রজেক্ট, দুটি tag (1.0 ও 2.0)।

আর্কিটেকচার:
v1 চলছে (Up, /health ok) → v2 image build → docker stop v1 → docker run v2 → যাচাই (ok) → v2 ভাঙা হলে → docker stop v2 → docker run v1 (rollback) → আবার ok ✅

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# v1 থেকে v2 বানান: server.js-এ লেখা বদলান
# (server.js-এ '/' রুটের লেখা "Hello v2" করুন)
docker build -t technova-api:2.0 ./server
docker images technova-api

# v1 চালান, তারপর v2-তে আপডেট করুন
docker run -d --name technova-api -p 3000:3000 technova-api:1.0
curl -s http://localhost:3000/
docker stop technova-api && docker rm technova-api
docker run -d --name technova-api -p 3000:3000 technova-api:2.0
curl -s http://localhost:3000/
# নমুনা আউটপুট — চিত্রণমূলক: Hello v2

# ভাঙা v3 বানিয়ে rollback পরীক্ষা করুন (ইচ্ছা করে ভুল)
# server.js-এ syntax ভুল করুন, v3 build করুন, চালান — container Exited হবে
docker build -t technova-api:3.0 ./server || echo "build failed - good, v3 is broken"
docker run -d --name technova-api-v3 -p 3005:3000 technova-api:3.0 || true
docker ps -a --filter name=technova-api-v3
# Rollback: v3 সরিয়ে v2 আবার চালান
docker rm -f technova-api-v3 || true
docker stop technova-api && docker rm technova-api
docker run -d --name technova-api -p 3000:3000 technova-api:2.0
curl -s http://localhost:3000/
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: `server.js`-এ রুটের লেখা বদলে `technova-api:2.0` build করুন; `docker images`-এ v1 ও v2 দুটোই দেখুন — পুরনো image মুছবেন না, rollback-এর জন্য লাগবে।

ধাপ 2: v1 চালিয়ে `curl /` দিয়ে লেখা দেখুন, তারপর stop/rm করে v2 চালিয়ে আবার `curl` করুন — আপডেট সম্পূর্ণ।

ধাপ 3: ইচ্ছা করে `server.js`-এ syntax ভুল করে v3 build করুন — build ব্যর্থ হলে সেটিই ভালো (ভাঙা image deploy হয়নি); build সফল হলেও container Exited হবে, সেটিও দেখুন।

ধাপ 4: v3 সরিয়ে v2 আবার চালান — `curl /`-এ v2-র লেখা ফিরে আসে, এটাই rollback।

ধাপ 5: Compose-এ একই কাজ করুন: compose.yaml-এ `image: technova-api:2.0` লিখে `docker compose up -d` করুন, তারপর tag বদলে `1.0` করে আবার `up -d` করুন।

ধাপ 6: আপডেটের সময় downtime মাপুন: `curl` বারবার চালিয়ে দেখুন stop থেকে start পর্যন্ত কতক্ষণ সাড়া মেলে না — এই gap-ই blue-green/canary deployment কমায় (বইয়ের অধ্যায় ২৩ দেখুন)।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker images technova-api --format 'table {{.Tag}}\t{{.Size}}'
curl -s http://localhost:3000/
# প্রত্যাশিত: v2-র লেখা আসে; rollback-এর পরও v2-র লেখা আসে;
# v1 image এখনো আছে — rollback সম্ভব।
```

প্রত্যাশিত ফলাফল: v1 থেকে v2-তে আপডেট হয়, ভাঙা v3 deploy হয় না বা চালু হয় না, v2-তে rollback করে আবার সাড়া মেলে; পুরনো image রেখে দেওয়াই rollback-এর চাবি।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
আপডেটের পর পুরনো লেখাই আসছে	পুরনো container এখনো চলছে — `docker ps` দেখে নাম ও image মিলিয়ে দেখুন; একই নামে নতুন container চালাতে আগে পুরনোটি stop/rm করতে হয়।
Rollback-এর পর image পাওয়া যাচ্ছে না	`docker rmi` দিয়ে পুরনো image মুছে ফেলেছেন — rollback-এর জন্য আগের tag-এর image রাখতে হয়, অথবা registry থেকে pull করতে হয়।
Compose-এ tag বদলেও নতুন image আসছে না	`docker compose up -d`-এর আগে `docker compose pull api` বা `--build` দিন — না হলে স্থানীয় পুরনো image-ই ব্যবহার হয়।
আপডেটে ডেটা হারিয়ে গেল	অ্যাপের image বদলেছেন, কিন্তু ডেটা volume-এ আছে কি না দেখুন — image বদলালে volume-এর ডেটা যায় না, যদি `down -v` না দিয়ে থাকেন।

Troubleshooting

আপডেট ব্যর্থ হলে প্রথমে `docker logs <নতুন container>` দেখুন — নতুন সংস্করণের ভুল সেখানেই থাকে। তারপর `docker ps -a` দিয়ে দেখুন নতুন container Exited কি না; Exited হলে rollback করুন, আটকে থাকবেন না। Rollback-এর আগে নতুন সংস্করণের image-টি রেখে দিন — পরে তদন্তে লাগবে।

Cleanup

`docker stop technova-api && docker rm technova-api`; v1 ও v2 image রাখুন। `server.js`-এর ভাঙা v3 কোডটি ঠিক করে ফেলুন — পরের ল্যাবে পরিষ্কার কোড লাগবে।

Reflection প্রশ্ন

১. Rollback-এর জন্য পুরনো image রাখা কেন জরুরি — `:latest` tag ব্যবহার করলে rollback কেন কঠিন হয়? ২. আপডেটের downtime কমাতে কী কী কৌশল আছে (দুটি container, blue-green)? ৩. ডেটাবেসের schema বদলালে শুধু image rollback-ই কি যথেষ্ট — কেন নয়?

ল্যাব 24

প্রোডাকশন Container Incident সমস্যা সমাধান করা

উদ্দেশ্য (Objective): একটি অনুকরণ করা (simulated) প্রোডাকশন incident — TechNova-র API হঠাৎ সাড়া দিচ্ছে না — তদন্ত করে কারণ খুঁজে বের করা, ঠিক করা, ও incident report লেখা। এই ল্যাবটি পুরো বইয়ের পরীক্ষা: আগের ২৩টি ল্যাবের সব দক্ষতা একসাথে ব্যবহার হবে।

পূর্বশর্ত (Prerequisites): ল্যাব ১–২৩ সম্পূর্ণ; Compose MERN পরিবেশ চালাতে পারেন।

প্রয়োজনীয় পরিবেশ (Required Environment): Docker Engine + Compose v2, technova-app প্রজেক্ট, একটি নোট ফাইল (incident timeline লেখার জন্য)।

আর্কিটেকচার:
ব্যবহারকারী: app.technova.example সাড়া দিচ্ছে না → তদন্ত: nginx Up? → api Up? → mongo Up? → log → disk পূর্ণ? → OOM? → network? → কারণ খুঁজে ঠিক করা → যাচাই → incident report

সম্পূর্ণ কনফিগারেশন ও কমান্ড

```
# --- Incident অনুকরণ: তিনটি ভিন্ন পরিস্থিতি, একে একে পরীক্ষা করুন ---
# পরিস্থিতি A: api container থেমে গেছে (crash অনুকরণ)
docker compose up -d --build
docker compose stop api
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/health
# নমুনা আউটপুট — চিত্রণমূলক: 502 (nginx চলছে, api নেই)

# তদন্তের কমান্ড (প্রতিটি incident-এ এই ক্রমে চালান)
docker compose ps
docker compose logs --tail 50 api
docker compose logs --tail 20 nginx
docker stats --no-stream
docker system df
docker exec technova-mongo mongosh -u admin -p CHANGE-ME-MONGO-PASSWORD --eval
'db.adminCommand("ping")' || echo "mongo unreachable"
docker network inspect technova-app_technova-net 2>/dev/null || docker network ls

# পরিস্থিতি B: disk পূর্ণ (log-এ দেখুন — অনুকরণের জন্য বড় ফাইল লিখবেন না,
# বরং docker system df দিয়ে disk ব্যবহার দেখে সিদ্ধান্ত নিন)
# পরিস্থিতি C: ভুল env (MONGO_PASSWORD বদলে api চালু — সংযোগ ব্যর্থ হবে)
# ঠিক করা: docker compose restart api / docker compose up -d
curl -s http://localhost:8080/health
```

ধাপে ধাপে নির্দেশনা

ধাপ 1: Compose পরিবেশ চালু করুন (`docker compose up -d --build`), তারপর `curl http://localhost:8080/health` দিয়ে স্বাভাবিক অবস্থা যাচাই করুন ও সময় লিখুন — এটাই incident-এর "আগের" অবস্থা।

ধাপ 2: পরিস্থিতি A অনুকরণ করুন: `docker compose stop api` দিয়ে api থামান, ব্রাউজার/curl-এ 502 দেখুন, তারপর তদন্ত শুরু করুন — `docker compose ps` দিয়ে কোন সার্ভিস থেমে আছে তা খুঁজুন।

ধাপ 3: `docker compose logs --tail 50 api` দিয়ে শেষ log পড়ুন — crash-এর কারণ (যেমন MongoDB সংযোগ ব্যর্থ) সেখানে থাকবে; `docker compose logs nginx`-এ 502-র দাগ দেখুন।

ধাপ 4: `docker stats --no-stream` ও `docker system df` দিয়ে resource ও disk দেখুন — OOM বা disk পূর্ণ কি না বাদ দিন।

ধাপ 5: কারণ নিশ্চিত করে ঠিক করুন: `docker compose restart api` বা `docker compose up -d` দিয়ে api আবার চালান, `curl /health` দিয়ে যাচাই করুন।

ধাপ 6: পরিস্থিতি C অনুকরণ করুন: `.env`-এ ভুল password দিয়ে `docker compose up -d` করুন — api-র log-এ authentication error দেখুন, তারপর সঠিক password দিয়ে আবার চালান।

ধাপ 7: Incident report লিখুন: কখন শুরু, কখন ধরা পড়ল, লক্ষণ, তদন্তের ধাপ, মূল কারণ, সমাধান, কতক্ষণ বন্ধ ছিল, ভবিষ্যতে কী বদলাবেন (যেমন health check alert, `restart: unless-stopped`)।

যাচাই (Verification) ও প্রত্যাশিত ফলাফল

```
docker compose ps --format 'table {{.Name}}\t{{.Status}}'
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/health
# প্রত্যাশিত: সব সার্ভিস Up (healthy); HTTP 200 ফিরে আসে।
# নমুনা আউটপুট — চিত্রণমূলক: 200
```

প্রত্যাশিত ফলাফল: Incident-এর লক্ষণ (502) থেকে `docker compose ps` ও log দিয়ে কারণ (api থেমে যাওয়া / ভুল password) খুঁজে পাওয়া যায়, restart/up দিয়ে সাড়া ফিরে আসে (HTTP 200), আর incident report-এ timeline, কারণ ও প্রতিরোধের উপায় লেখা থাকে।

সাধারণ ভুল ও সমাধান

ভুল / লক্ষণ	সমাধান
502 দেখে nginx-কে দোষ দেওয়া	502 মানে nginx চলছে, কিন্তু পেছনের api পৌঁছাচ্ছে না — প্রথমে api-র অবস্থা দেখুন, nginx-এর config নয়।
Log-এ কিছু নেই, container মুছে ফেলেছি	তদন্তের আগে container মুছবেন না — মুছলে log-ও যায়; আগে `docker logs` ও `docker inspect` সংগ্রহ করুন।
Restart করলেও আবার থেমে যাচ্ছে	মূল কারণ ঠিক হয়নি — log-এর শেষ error পড়ুন; বারবার restart করা (crash loop) ঢাকার উপায় নয়, কারণ খুঁজুন।
Incident-এর সময় কেউ কিছু লেখেনি	Timeline না থাকলে পরে কারণ বোঝা যায় না — incident চলাকালীনই সময়সহ নোট লিখুন, এটাই runbook-এর অংশ।

Troubleshooting

Incident-এ শান্ত থাকুন ও ক্রম মেনে চলুন: (১) `docker compose ps` — কে থেমে আছে, (২) `docker compose logs` — কেন থেমেছে, (৩) `docker stats` / `docker system df` — resource ঠিক আছে কি না, (৪) `docker exec` দিয়ে ভেতর থেকে পরীক্ষা, (৫) ঠিক করুন, (৬) যাচাই করুন, (৭) report লিখুন। একসাথে অনেক কিছু বদলাবেন না — একটি বদল, একটি যাচাই।

Cleanup

`docker compose down` দিয়ে পরিবেশ থামান (volume মুছবেন না)। Incident report-টি সংরক্ষণ করুন — ক্যাপস্টোনে এটিই runbook-এর অংশ হবে।

Reflection প্রশ্ন

১. 502, 503 ও connection refused — প্রতিটি কী বলছে, আর তদন্তের প্রথম ধাপ কোনটি? ২. Crash loop-এ বারবার restart করা কেন ভুল — কখন restart ঠিক, কখন কারণ খোঁজা জরুরি? ৩. Incident report-এ কী কী থাকা উচিত, যাতে পরের বার একই incident দ্রুত সামলানো যায়?

ব্যবহারিক অ্যাসাইনমেন্ট

১৫টি ব্যবহারিক অ্যাসাইনমেন্ট

প্রতিটি অ্যাসাইনমেন্ট একা করুন — ল্যাবগুলো শেষ করার পর। ডেলিভারেবল জমা দেওয়ার সময় কমান্ড, আউটপুট ও সিদ্ধান্তের যুক্তি একসাথে দিন। Secret বা আসল credential কখনো জমা দেবেন না — কাল্পনিক মান ব্যবহার করুন।

অ্যাসাইনমেন্ট 1 — VM বনাম Container: সংখ্যায় তুলনা প্রতিবেদন

উদ্দেশ্য: একটি ছোট ওয়েব সার্ভার VM ও container-এ চালিয়ে শুরু হওয়ার সময়, মেমরি, CPU ও disk ব্যবহার মেপে একটি তুলনা প্রতিবেদন লেখা।

কাজ: 1. VM ও container-এ একই nginx সার্ভার চালিয়ে তিনবার করে শুরু হওয়ার সময় মাপুন। 2. মেমরি ও disk ব্যবহারের সংখ্যা সংগ্রহ করুন। 3. কোন কাজে VM ভালো, কোন কাজে container ভালো — তিনটি করে উদাহরণসহ সিদ্ধান্ত লিখুন।

ডেলিভারেবল: তুলনা টেবিলসহ ২–৩ পাতার প্রতিবেদন (সংখ্যাসহ)। পরিমাপের কমান্ড ও আউটপুটের স্ক্রিনশট/কপি।

মূল্যায়নের মানদণ্ড: সংখ্যা সঠিক ও পুনরাবৃত্তিযোগ্য (৩০%), তুলনার যুক্তি স্পষ্ট (৪০%), VM/container বেছে নেওয়ার সিদ্ধান্ত যুক্তিসংগত (৩০%)।

অ্যাসাইনমেন্ট 2 — Docker Architecture চিত্র ও ব্যাখ্যা

উদ্দেশ্য: Docker client, daemon, containerd, runc ও registry-র সম্পর্ক নিজের আঁকা চিত্রসহ ব্যাখ্যা করা।

কাজ: 1. নিজের ভাষায় একটি architecture চিত্র আঁকুন। 2. `docker run hello-world`-এর প্রতিটি ধাপ চিত্রে চিহ্নিত করুন। 3. Socket নিরাপত্তার ঝুঁকি এক অনুচ্ছেদে লিখুন।

ডেলিভারেবল: এক পাতার চিত্র + এক পাতার ব্যাখ্যা।

মূল্যায়নের মানদণ্ড: চিত্রের প্রতিটি অংশ সঠিক (৪০%), ধাপের ব্যাখ্যা সম্পূর্ণ (৪০%), নিরাপত্তা আলোচনা আছে (২০%)।

অ্যাসাইনমেন্ট 3 — নতুন মেশিনে Docker ইনস্টলেশন গাইড

উদ্দেশ্য: একটি ফাঁকা Ubuntu 24.04 VM-এ Docker Engine ও Compose ইনস্টল করে ধাপে ধাপে গাইড লেখা।

কাজ: 1. অফিসিয়াল repository থেকে ইনস্টল করুন। 2. Non-root user-এ যাচাই করুন। 3. সাধারণ তিনটি ভুল ও সমাধান গাইডে যোগ করুন।

ডেলিভারেবল: ইনস্টলেশন গাইড (কমান্ডসহ) + যাচাইয়ের আউটপুট।

মূল্যায়নের মানদণ্ড: ধাপগুলো সম্পূর্ণ ও ক্রমানুসারে (৫০%), যাচাই সফল (৩০%), ভুলের সমাধান সঠিক (২০%)।

অ্যাসাইনমেন্ট 4 — নিজের স্ট্যাটিক সাইটের Dockerfile

উদ্দেশ্য: নিজের একটি HTML/CSS সাইটের জন্য Dockerfile লিখে image বানানো ও চালানো।

কাজ: 1. Dockerfile লিখুন (FROM, COPY, EXPOSE, CMD)। 2. Image build করে চালিয়ে ব্রাউজারে দেখান। 3. `.dockerignore` যোগ করুন।

ডেলিভারেবল: Dockerfile + build log + ব্রাউজার স্ক্রিনশট।

মূল্যায়নের মানদণ্ড: Dockerfile সঠিক (৪০%), image চলে (৩০%), .dockerignore যুক্তিসংগত (৩০%)।

অ্যাসাইনমেন্ট 5 — Node.js অ্যাপ Containerize করা

উদ্দেশ্য: TechNova API-র জন্য production-উপযোগী Dockerfile লেখা।

কাজ: 1. `npm ci --omit=dev` ব্যবহার করুন। 2. Non-root user যোগ করুন। 3. `.dockerignore`-এ `.env` ও `node_modules` রাখুন।

ডেলিভারেবল: Dockerfile + `curl /health` যাচাই।

মূল্যায়নের মানদণ্ড: Build সফল ও API চলে (৪০%), non-root যাচাই (৩০%), secret image-এ নেই (৩০%)।

অ্যাসাইনমেন্ট 6 — Build Cache অপ্টিমাইজেশন পরীক্ষা

উদ্দেশ্য: Dockerfile-এর নির্দেশের ক্রম বদলে build সময়ের পার্থক্য মাপা।

কাজ: 1. ভুল ক্রমে (COPY আগে) ও সঠিক ক্রমে build সময় মাপুন। 2. তিনবার করে গড় নিন। 3. ফলাফল টেবিলে লিখুন।

ডেলিভারেবল: পরীক্ষার টেবিল + সিদ্ধান্ত।

মূল্যায়নের মানদণ্ড: পরিমাপ পদ্ধতি সঠিক (৪০%), ফলাফল ব্যাখ্যা করা হয়েছে (৪০%), সঠিক ক্রম বেছে নেওয়া হয়েছে (২০%)।

অ্যাসাইনমেন্ট 7 — Multi-stage Build তুলনা

উদ্দেশ্য: একই অ্যাপের single-stage ও multi-stage image-এর আকার ও নিরাপত্তা তুলনা করা।

কাজ: 1. দুটি Dockerfile লিখুন। 2. আকার, layer সংখ্যা ও ভেতরের টুল তুলনা করুন। 3. কোনটি প্রোডাকশনে নেবেন — যুক্তি লিখুন।

ডেলিভারেবল: দুটি Dockerfile + তুলনা টেবিল।

মূল্যায়নের মানদণ্ড: দুটি image-ই চলে (৪০%), আকারের পার্থক্য ব্যাখ্যা করা হয়েছে (৩০%), নিরাপত্তা আলোচনা আছে (৩০%)।

অ্যাসাইনমেন্ট 8 — Image আকার অর্ধেক করা

উদ্দেশ্য: একটি বড় image-কে (500 MB+) কৌশল প্রয়োগ করে অর্ধেক বা তার কম আকারে আনা।

কাজ: 1. Base image, layer একত্র করা, cache পরিষ্কার — প্রতিটি কৌশলের প্রভাব মাপুন। 2. প্রতিটি ধাপের আগে-পরের আকার লিখুন। 3. চূড়ান্ত image চালিয়ে যাচাই করুন।

ডেলিভারেবল: আগে-পরের আকারের টেবিল + চূড়ান্ত Dockerfile।

মূল্যায়নের মানদণ্ড: আকার কমেছে (৪০%), প্রতিটি কৌশলের প্রভাব আলাদা করে দেখানো হয়েছে (৩০%), image এখনো চলে (৩০%)।

অ্যাসাইনমেন্ট 9 — Docker Hub-এ Publish ও Pull

উদ্দেশ্য: নিজের image Docker Hub-এ push করে অন্য মেশিন (বা VM) থেকে pull করে চালানো।

কাজ: 1. Repository private রাখুন। 2. Semantic tag (1.0.0) ও digest লিখে রাখুন। 3. Pull করে চালিয়ে যাচাই করুন।

ডেলিভারেবল: Hub repository-র লিংক + pull/run যাচাই।

মূল্যায়নের মানদণ্ড: Push/pull সফল (৫০%), tag কৌশল সঠিক (২৫%), private/public সিদ্ধান্ত যুক্তিসংগত (২৫%)।

অ্যাসাইনমেন্ট 10 — Custom Network দিয়ে দুটি সার্ভিস যুক্ত করা

উদ্দেশ্য: API ও frontend container-কে custom network-এ যুক্ত করে নামে যোগাযোগ করানো।

কাজ: 1. Custom bridge network বানান। 2. নামে `curl` করে যাচাই করুন। 3. ভিন্ন network-এর container থেকে পৌঁছানো যায় না — প্রমাণ করুন।

ডেলিভারেবল: Network inspect + যাচাইয়ের আউটপুট।

মূল্যায়নের মানদণ্ড: নামে যোগাযোগ কাজ করে (৫০%), বিচ্ছিন্নতা প্রমাণিত (৩০%), ব্যাখ্যা স্পষ্ট (২০%)।

অ্যাসাইনমেন্ট 11 — Volume দিয়ে ডেটাবেস Backup ও Restore

উদ্দেশ্য: MongoDB-র ডেটা volume-এ রেখে backup নিয়ে নতুন container-এ restore করা।

কাজ: 1. `mongodump` দিয়ে backup নিন। 2. Volume মুছে নতুন container-এ `mongorestore` করুন। 3. ডেটা ফিরে এসেছে — যাচাই করুন।

ডেলিভারেবল: Backup/restore ধাপ + যাচাই।

মূল্যায়নের মানদণ্ড: Backup সফল (৩০%), restore-এ ডেটা ফিরেছে (৪০%), ধাপগুলো লেখা আছে (৩০%)।

অ্যাসাইনমেন্ট 12 — Compose দিয়ে পূর্ণ MERN পরিবেশ

উদ্দেশ্য: Frontend, backend, MongoDB ও Nginx — চার সার্ভিসের Compose ফাইল লেখা।

কাজ: 1. Health check ও `depends_on: service_healthy` যোগ করুন। 2. শুধু nginx-এর port বাইরে খুলুন। 3. `down` করে আবার `up` করলে ডেটা থাকে — প্রমাণ করুন।

ডেলিভারেবল: compose.yaml + যাচাইয়ের আউটপুট।

মূল্যায়নের মানদণ্ড: সব সার্ভিস healthy (৪০%), network/volume সঠিক (৩০%), ডেটা টিকে থাকে (৩০%)।

অ্যাসাইনমেন্ট 13 — Environment আলাদা করা (dev বনাম prod)

উদ্দেশ্য: একই Compose ফাইল দিয়ে dev ও production পরিবেশের পার্থক্য দেখানো।

কাজ: 1. `.env` ও override ফাইল ব্যবহার করুন। 2. Dev-এ bind mount, prod-এ image — পার্থক্য লিখুন। 3. Secret কোথাও hardcode নেই — প্রমাণ করুন।

ডেলিভারেবল: দুটি পরিবেশের কনফিগারেশন + পার্থক্যের টেবিল।

মূল্যায়নের মানদণ্ড: দুটি পরিবেশই চলে (৪০%), পার্থক্য ব্যাখ্যা করা হয়েছে (৩০%), secret নিরাপদ (৩০%)।

অ্যাসাইনমেন্ট 14 — নিরাপত্তা Audit: নিজের Image পরীক্ষা করা

উদ্দেশ্য: নিজের একটি image-কে root user, secret, পুরনো base ও অপ্রয়োজনীয় টুলের দিক থেকে audit করা।

কাজ: 1. `whoami`, `docker history` ও `docker inspect` দিয়ে পরীক্ষা করুন। 2. পাওয়া প্রতিটি সমস্যার সমাধান লিখুন। 3. ঠিক করা image আবার audit করুন।

ডেলিভারেবল: Audit প্রতিবেদন + ঠিক করা Dockerfile।

মূল্যায়নের মানদণ্ড: সমস্যা খুঁজে পাওয়া গেছে (৪০%), সমাধান সঠিক (৪০%), পুনরায় যাচাই করা হয়েছে (২০%)।

অ্যাসাইনমেন্ট 15 — Incident Report লেখা

উদ্দেশ্য: একটি অনুকরণ করা incident (API down) তদন্ত করে পূর্ণ incident report লেখা।

কাজ: 1. Timeline, লক্ষণ, তদন্তের ধাপ, মূল কারণ, সমাধান লিখুন। 2. ভবিষ্যতে প্রতিরোধের দুটি উপায় লিখুন। 3. Runbook-এ একটি ধাপ যোগ করুন।

ডেলিভারেবল: Incident report (১–২ পাতা)।

মূল্যায়নের মানদণ্ড: Timeline সম্পূর্ণ (৩০%), মূল কারণ সঠিক (৪০%), প্রতিরোধের উপায় বাস্তবসম্মত (৩০%)।

মেজর প্রজেক্ট ১

প্রজেক্ট ১ — Production Node.js Container

উদ্দেশ্য: TechNova-র Express API-র জন্য একটি সত্যিকারের production-উপযোগী container বানানো — যা ছোট, নিরাপদ, health check দেয়, environment দিয়ে কনফিগার হয়, আর আপডেট করা যায়।

আর্কিটেকচার:
Source (server.js + package.json) → Multi-stage build (build stage: node:20 → runtime stage: node:20-alpine) → Image technova-api:1.0.0 → Container (USER node, memory 512m, CPU 0.5, healthcheck /health) → Host :3000 → curl /health

প্রয়োজনীয়তা

Multi-stage Dockerfile: build stage-এ `npm ci` (dev dependency-সহ, build-এর জন্য), runtime stage-এ শুধু production dependency (`npm ci --omit=dev`) ও `COPY --from=build` দিয়ে দরকারি ফাইল। **Non-root user:** `USER node`, `COPY --chown=node:node`। **Production dependency:** devDependency image-এ থাকবে না — `docker exec` দিয়ে যাচাই করুন। **Environment config:** port, log level ও API message environment variable থেকে আসবে; `.env` ফাইল image-এ যাবে না। **Health check:** Dockerfile-এ `HEALTHCHECK` (wget /health) অথবা Compose-এ `healthcheck:` — interval 10s, retries 3। **Optimization:** base image alpine, layer একত্র করা, `npm cache clean --force`, `.dockerignore` — চূড়ান্ত image 200 MB-এর নিচে আনার লক্ষ্য রাখুন (আপনার dependency অনুযায়ী সংখ্যা আলাদা হতে পারে)।

সম্পূর্ণ কনফিগারেশন

```
# Dockerfile (production)
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . ./
FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=build --chown=node:node /app/server.js ./server.js
USER node
EXPOSE 3000
HEALTHCHECK --interval=10s --timeout=5s --retries=3 \
  CMD wget -q -O /dev/null http://localhost:3000/health || exit 1
CMD ["node", "server.js"]

# build ও চালান
docker build -t technova-api:1.0.0 .
docker run -d --name technova-api-prod --memory 512m --cpus 0.5 \
  -e NODE_ENV=production -e PORT=3000 -p 3000:3000 technova-api:1.0.0
```

```
curl -s http://localhost:3000/health
# নমুনা আউটপুট — চিত্রণমূলক: {"status":"ok","app":"technova-api"}
```

ধাপে ধাপে নির্দেশনা

ধাপ ১: উপরের Dockerfile লিখুন ও build করুন। **ধাপ ২:** `docker images` দিয়ে আকার লিখুন ও ল্যাব ৮-এর image-এর সঙ্গে তুলনা করুন। **ধাপ ৩:** Resource limit দিয়ে চালিয়ে `docker stats` ও `docker inspect --format '{{.State.Health.Status}}'` দিয়ে health যাচাই করুন। **ধাপ ৪:** `whoami` দিয়ে non-root যাচাই করুন, `ls /app/node_modules/.bin` দিয়ে dev tool নেই তা যাচাই করুন। **ধাপ ৫:** `.env` বদলে (যেমন API_MESSAGE) container আবার চালিয়ে আচরণ বদলায় কি না দেখুন। **ধাপ ৬:** v1.0.1 tag দিয়ে একটি ছোট আপডেট build করে ল্যাব ২৩-এর পদ্ধতিতে আপডেট ও rollback পরীক্ষা করুন।

যাচাই ও প্রত্যাশিত ফলাফল

Image 200 MB-এর কাছাকাছি বা নিচে, container `node` user-এ চলে, `/health` ok দেয়, `docker inspect`-এ Health `healthy` দেখায়, resource limit `docker stats`-এ দেখা যায়, dev tool image-এ নেই।

সাধারণ ভুল ও সমাধান

ভুল	সমাধান
Health check ব্যর্থ	alpine-এ `wget` আছে কি না দেখুন; `HEALTHCHECK`-এর URL-এ `localhost` ব্যবহার করুন, container-এর নাম নয়।
Non-root-এ EACCES	`COPY --chown=node:node` দিন; `USER` লাইন `RUN npm ci`-এর পরে রাখুন।
Image আশানুরূপ ছোট হয়নি	`docker history` দিয়ে বড় layer খুঁজুন; runtime stage-এ dev dependency ঢুকেছে কি না দেখুন।

Cleanup

`docker stop technova-api-prod && docker rm technova-api-prod`; image রাখুন — প্রজেক্ট ২-এ ব্যবহার হবে।

মূল্যায়ন

Multi-stage সঠিক (২৫%), non-root ও নিরাপত্তা (২০%), health check কাজ করে (২০%), আকার optimized (১৫%), env config সঠিক (১০%), আপডেট/rollback পরীক্ষা করা হয়েছে (১০%)।

মেজর প্রজেক্ট ২

প্রজেক্ট ২ — Complete MERN Docker Environment

উদ্দেশ্য: TechNova-র পূর্ণ MERN অ্যাপ — React frontend, Express backend, MongoDB ও Nginx — একটি Compose ফাইলে চালানো, dev ও production দুটি ভিন্ন রূপে, যাতে প্রতিটি পার্থক্যের কারণ ব্যাখ্যা করতে পারেন।

আর্কিটেকচার:
Host :8080 → nginx (reverse proxy, একমাত্র বাইরের port) → client (React, nginx দিয়ে পরিবেশিত static build) + api (Express :3000) → mongo (volume mongo-data, শুধু internal network)
Network: frontend-net (nginx ↔ client/api) + backend-net (api ↔ mongo) — mongo শুধু backend-net-এ, বাইরে থেকে পৌঁছানো যায় না

প্রয়োজনীয়তা

Frontend: React/Vite অ্যাপ multi-stage build-এ `dist/` তৈরি করে nginx stage-এ পরিবেশন করবে, অথবা dev-এ Vite dev server চলবে। **Backend:** প্রজেক্ট ১-এর production image (non-root, health check, resource limit)। **MongoDB:** named volume `mongo-data`-তে `/data/db`, health check `mongosh ping`, শুধু backend network-এ। **Nginx:** reverse proxy — `/` frontend-এ, `/api/` backend-এ proxy করবে। **Custom network:** দুটি network — `frontend-net` ও `backend-net`; api দুটোতেই থাকে, mongo শুধু backend-net-এ। **Persistent storage:** `mongo-data` volume; `docker compose down`-এর পরেও ডেটা থাকে, `down -v` দিলে যায় — এই পার্থক্য নথিভুক্ত করুন। **Env config:** `.env` ফাইলে MONGO_USER, MONGO_PASSWORD (কাল্পনিক), NODE_ENV; Compose-এ `${...}` substitution। **Health checks:** তিনটি সার্ভিসেই health check; `depends_on: condition: service_healthy` দিয়ে ক্রম ঠিক করুন।

Dev বনাম Production — পার্থক্য ব্যাখ্যা

বিষয়	Development	Production
Frontend	Vite dev server, hot reload, bind mount (`./client:/app`) — কোড বদলালে সঙ্গে সঙ্গে দেখা যায়	Multi-stage build-এ `dist/`, nginx দিয়ে static পরিবেশন, bind mount নেই
Backend	`nodemon`, bind mount, বিস্তারিত log	`node server.js`, non-root, resource limit, restart: unless-stopped
Port	3000, 5173 (Vite) হোস্টে খোলা থাকতে পারে (পরীক্ষার জন্য)	শুধু 80/443 (nginx) বাইরে খোলা; api ও mongo internal
Image	Build context থেকে সরাসরি, tag `:dev`	Registry থেকে pull করা, নির্দিষ্ট tag (যেমন `:1.0.0`), digest লিখে রাখা
Volume	Bind mount বেশি (কোড), volume কম	Named volume (ডেটা), bind mount প্রায় নেই — হোস্ট mount ঝুঁকি এড়াতে

সতর্কতা — host mount ঝুঁকি: Production-এ হোস্টের ফোল্ডার bind mount করবেন না — হোস্টের ফাইল container-এ, আর container-এর লেখা হোস্টে পৌঁছে যায়; ভুল mount-এ হোস্টের গুরুত্বপূর্ণ ফোল্ডার মুছে যেতে পারে। Production-এ named volume ব্যবহার করুন। Dev-এ bind mount শুধু নিজের কোড ফোল্ডারে, সীমিতভাবে ব্যবহার করুন।

সম্পূর্ণ কনফিগারেশন

```
# compose.yaml — production রূপ (dev-এর জন্য compose.override.yaml দেখুন নিচে)
services:
  mongo:
    image: mongo:7
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER:-admin}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD:-CHANGE-ME-MONGO-PASSWORD}
    volumes: [mongo-data:/data/db]
    networks: [backend-net]
    healthcheck:
      test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')", "-u", "${MONGO_USER:-
admin}", "-p", "${MONGO_PASSWORD:-CHANGE-ME-MONGO-PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s
  api:
    image: technova-api:1.0.0
    restart: unless-stopped
    environment:
      NODE_ENV: production
      MONGO_URI: mongodb://${MONGO_USER:-admin}:${MONGO_PASSWORD:-CHANGE-ME-MONGO-PASSWORD}
@mongo:27017/technova?authSource=admin
    depends_on:
      mongo: { condition: service_healthy }
    networks: [frontend-net, backend-net]
    deploy:
      resources:
        limits: { cpus: '0.50', memory: 512M }
    healthcheck:
      test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://localhost:3000/health"]
      interval: 10s
      timeout: 5s
      retries: 5
  nginx:
    image: nginx:1.27-alpine
    restart: unless-stopped
    ports: ["8080:80"]
    volumes: ["./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro"]
    depends_on:
      api: { condition: service_healthy }
    networks: [frontend-net]
volumes:
  mongo-data:
networks:
  frontend-net:
  backend-net:
```

```
# dev-এর জন্য compose.override.yaml: api-তে bind mount ও nodemon, client সার্ভিস যোগ করুন;
# production-এ চালাতে: docker compose -f compose.yaml up -d (override ফাইল ছাড়া)
```

ধাপে ধাপে নির্দেশনা

ধাপ ১: উপরের compose.yaml লিখুন ও `docker compose up -d --build` চালান। **ধাপ ২:** `docker compose ps`-এ সব সার্ভিস `(healthy)` দেখুন; `curl http://localhost:8080/health` ও ব্রাউজারে frontend দেখুন। **ধাপ ৩:** `docker compose exec api getent hosts mongo` দিয়ে DNS যাচাই করুন; `docker compose exec nginx wget -qO- http://api:3000/health` দিয়ে internal যোগাযোগ যাচাই করুন। **ধাপ ৪:** MongoDB-তে ডেটা লিখে `docker compose down` করে আবার `up` করুন — ডেটা ফিরে আসে কি না দেখুন। **ধাপ ৫:** Dev রূপ চালান (override ফাইলসহ) — কোড বদলে bind mount-এ সঙ্গে সঙ্গে প্রতিফলন দেখুন। **ধাপ ৬:** Production রূপে ফিরে গিয়ে `docker compose config` দিয়ে দুটি রূপের পার্থক্য নথিভুক্ত করুন।

যাচাই ও প্রত্যাশিত ফলাফল

সব সার্ভিস healthy, frontend ব্রাউজারে আসে, `/api/health` ok দেয়, mongo শুধু backend-net-এ আছে (`docker network inspect`-এ যাচাই করুন), `down/up`-এর পর ডেটা থাকে, dev-এ bind mount কাজ করে, production-এ bind mount নেই।

মূল্যায়ন

চারটি সার্ভিস চলে (২৫%), দুটি network সঠিকভাবে আলাদা (২০%), health check ও dependency (১৫%), persistent storage (১৫%), dev/prod পার্থক্য ব্যাখ্যা করা হয়েছে (১৫%), env/secret নিরাপদ (১০%)।

মেজর প্রজেক্ট ৩

প্রজেক্ট ৩ — TechNova Container Platform

উদ্দেশ্য: TechNova-র জন্য একটি ছোট কিন্তু সম্পূর্ণ container platform-এর নকশা ও বাস্তবায়ন — image build থেকে registry, deployment, storage, নিরাপত্তা, log, backup ও rollback পর্যন্ত, সঙ্গে operational documentation। এটি একটি একক অ্যাপ নয়, একটি প্ল্যাটফর্মের নীলনকশা।

আর্কিটেকচার:
Developer → Git push → Build (multi-stage, tag v1.2.3 + digest) → Private Registry (yourname/technova-api: 1.2.3) → Deploy host (docker compose pull + up) → Services (frontend-net / backend-net, non-root, limits, health checks) → Volume mongo-data (+ nightly backup) → Logs (json-file, max-size 10m) → Runbook (deploy / rollback / incident ধাপ)

প্রয়োজনীয়তা

Image build ও versioning: Semantic tag (`1.2.3`) + `latest` নয় — প্রতিটি release-এ নতুন tag, digest লিখে রাখা; `docker build -t yourname/technova-api:1.2.3`। **Private registry:** Docker Hub private repository (বা স্থানীয় registry container — `docker run -d -p 5000:5000 registry:2`); push/pull-এ login বাধ্যতামূলক। **Deployment config:** `compose.yaml` + `.env` (production মান), `restart: unless-stopped`, `depends_on: service_healthy`। **Persistent storage:** named volume `mongo-data`, backup-এর জন্য আলাদা volume বা bind mount (শুধু backup ফোল্ডার)। **Security controls:** non-root user, `--privileged` নিষিদ্ধ, Docker socket mount নিষিদ্ধ, secret শুধু `.env` /secret-এ (image-এ নয়), `read_only: true` যেখানে সম্ভব (api-র জন্য `tmpfs` সহ), resource limit প্রতিটি সার্ভিসে। **Resource limits:** api 512 MB / 0.5 CPU, mongo 1 GB / 1 CPU, nginx 128 MB / 0.25 CPU — `docker stats` দিয়ে যাচাই। **Logging:** json-file driver, `max-size: 10m`, `max-file: 3` ; log-এ secret ছাপা নিষিদ্ধ। **Backup ও recovery:** nightly `mongodump` (cron বা manual script), backup ফাইল আলাদা জায়গায় রাখা, restore পরীক্ষা করা (ল্যাব ১৭-এর মতো)। **Update ও rollback:** নতুন tag pull → `docker compose up -d` → যাচাই → ব্যর্থ হলে আগের tag-এ ফিরে যাওয়া (ল্যাব ২৩) — প্রতিটি ধাপ runbook-এ লেখা। **Operational documentation:** README-তে architecture চিত্র, সার্ভিস তালিকা, port তালিকা, env variable তালিকা, deploy ধাপ, rollback ধাপ, backup ধাপ, incident ধাপ — সব এক জায়গায়।

সম্পূর্ণ কনফিগারেশন (মূল অংশ)

```
# deploy.sh — TechNova Container Platform deploy script (lab-এ পরীক্ষার জন্য)
#!/usr/bin/env bash
set -euo pipefail
TAG="${1:-1.0.0}"
echo "Deploying technova-api:$TAG ..."
docker compose pull api
docker compose up -d
sleep 10
curl -fsS http://localhost:8080/health || { echo "Health check FAILED - rolling back"; docker
compose up -d; exit 1; }
echo "Deploy OK: $TAG healthy"
```

```
# backup.sh — MongoDB backup (nightly cron-এ চালানোর জন্য)
#!/usr/bin/env bash
set -euo pipefail
DATE=$(date +%F)
docker compose exec -T mongo mongodump -u admin -p CHANGE-ME-MONGO-PASSWORD --out /tmp/
backup-"$DATE" --quiet
echo "Backup done: /tmp/backup-$DATE"
# নমুনা আউটপুট — চিত্রণমূলক: Backup done

# compose.yaml-এ logging (প্রতিটি সার্ভিসে যোগ করুন)
#   logging:
#     driver: json-file
#     options: { max-size: "10m", max-file: "3" }
```

> **সতর্কতা — backup ও rollback:** Backup নেওয়া হয়নি এমন volume কখনো মুছবেন না। `docker compose down -v` production-এ কখনো চালাবেন না — `-v` মানে সব volume মুছে ফেলা। Rollback-এর আগে নতুন সংস্করণের log সংগ্রহ করুন — মুছে ফেললে কারণ জানা যাবে না।

ধাপে ধাপে নির্দেশনা

ধাপ ১: প্রজেক্ট ২-এর Compose পরিবেশকে ভিত্তি করে private registry-তে image push করুন। **ধাপ ২:** প্রতিটি সার্ভিসে resource limit ও logging limit যোগ করুন। **ধাপ ৩:** `deploy.sh` লিখে একটি আপডেট deploy করুন ও health check দিয়ে যাচাই করুন। **ধাপ ৪:** `backup.sh` দিয়ে backup নিয়ে নতুন volume-এ restore পরীক্ষা করুন। **ধাপ ৫:** ইচ্ছা করে ভাঙা tag deploy করে rollback পরীক্ষা করুন। **ধাপ ৬:** README (operational documentation) লিখুন — architecture, port, env, deploy, rollback, backup, incident — সব ধাপ। **ধাপ ৭:** ল্যাব ২৪-এর মতো একটি incident অনুকরণ করে runbook অনুযায়ী সামলান ও সময় মাপুন।

যাচাই ও প্রত্যাশিত ফলাফল

Image private registry-তে আছে, deploy script health check দিয়ে সফল হয়, backup থেকে restore-এ ডেটা ফিরে আসে, ভাঙা deploy-এ rollback কাজ করে, README-তে সব operational ধাপ আছে, resource limit ও log limit `docker inspect`-এ দেখা যায়।

মূল্যায়ন

Registry ও versioning (১৫%), deployment config (১৫%), নিরাপত্তা নিয়ন্ত্রণ (২০%), resource limit ও logging (১৫%), backup/recovery পরীক্ষা করা হয়েছে (১৫%), update/rollback (১০%), documentation সম্পূর্ণ (১০%)।

ক্যাপস্টোন প্রজেক্ট

ক্যাপস্টোন — TechNova-র DevOps Engineer হিসেবে সম্পূর্ণ Production MERN পরিবেশ

পরিস্থিতি: আপনি TechNova-র DevOps Engineer। TechNova-র MERN অ্যাপ (React frontend, Node.js/Express backend, MongoDB, Nginx) এখনো ডেভেলপারদের ল্যাপটপে চলে। আপনার কাজ: সেটিকে একটি সম্পূর্ণ containerized production পরিবেশে নিয়ে যাওয়া — যা নিরাপদ, পর্যবেক্ষণযোগ্য, backup-সহ, আর আপডেট/rollback করা যায়। এই ক্যাপস্টোনে আগের ২৪টি ল্যাব, ১৫টি অ্যাসাইনমেন্ট ও ৩টি প্রজেক্টের সব দক্ষতা একসাথে ব্যবহার হবে।

> **নিয়ম:** সব কাজ disposable VM বা আলাদা টেস্ট মেশিনে করুন। আসল secret, আসল ডোমেইন বা আসল সার্ভার ব্যবহার করবেন না — `technova.example`, `203.0.113.10`, `CHANGE-ME-...` ব্যবহার করুন। Solution guide-টি ক্যাপস্টোনের শেষে আছে — প্রথমে নিজে চেষ্টা করুন, তারপর মিলিয়ে দেখুন।

লক্ষ্য আর্কিটেকচার:

Internet → Host :80/8080 → nginx (frontend-net) → client (React static) + api (Express, non-root, 512m/0.5 CPU, healthcheck) → mongo (backend-net, volume mongo-data, healthcheck) | Registry (private, tag 1.0.0 + digest) → Deploy host | Backup (nightly mongodump) + Logs (max-size 10m) + Runbook

কী কী বানাতে হবে

১. Optimized frontend ও backend image: দুটি multi-stage Dockerfile — frontend-এ Vite build stage + nginx runtime stage, backend-এ প্রজেক্ট ১-এর production Dockerfile। দুটি image-ই 200 MB-এর কাছাকাছি বা নিচে আনার চেষ্টা করুন, আকার নথিভুক্ত করুন। **২. Multi-stage build:** build tool (npm, Vite) চূড়ান্ত image-এ থাকবে না — `docker exec` দিয়ে প্রমাণ করুন। **৩. Compose:** একটি `compose.yaml` — চারটি সার্ভিস, দুটি network (frontend-net, backend-net), named volume, health check, `depends_on: service_healthy`, restart policy, logging limit। **৪. Isolated service network:** mongo শুধু backend-net-এ; nginx শুধু frontend-net-এ; api দুটোতেই — `docker network inspect` দিয়ে প্রমাণ করুন। **৫. Persistent DB storage:** `mongo-data` volume; `down/up`-এর পর ডেটা থাকে। **৬. নিরাপদ env ও secret:** `.env` ফাইলে কাল্পনিক credential, `.gitignore` ও `.dockerignore`-এ `.env`, secret image layer-এ নেই (`docker history` দিয়ে যাচাই)। **৭. Health check:** তিনটি সার্ভিসেই health check, `docker compose ps`-এ `(healthy)`। **৮. Non-root নিরাপত্তা:** api ও client container non-root user-এ চলে; `--privileged` নেই, Docker socket mount নেই। **৯. Resource constraint:** প্রতিটি সার্ভিসে memory/CPU limit, `docker stats` দিয়ে যাচাই। **১০. Registry publish:** দুটি image-ই private registry-তে tag (`1.0.0`) ও digest-সহ push করা, pull করে চালিয়ে যাচাই। **১১. Deploy ও verify:** `deploy.sh` দিয়ে deploy, `curl /health` ও ব্রাউজারে frontend যাচাই। **১২. Backup ও recovery demo:** `mongodump` দিয়ে backup, নতুন volume-এ restore করে ডেটা ফিরে আসা দেখান। **১৩. Update ও rollback demo:** v1.1.0 deploy করে যাচাই, তারপর ভাঙা v1.2.0 deploy করে rollback করে v1.1.0-এ ফেরা — দুটোরই প্রমাণ রাখুন। **১৪. Production runbook:** deploy, rollback, backup, restore, incident (API down, mongo down, disk full) — প্রতিটির ধাপ, কমান্ড ও যাচাই একসাথে লেখা একটি runbook।

Acceptance Criteria (গ্রহণযোগ্যতার শর্ত)

#	শর্ত	যাচাই
১	`docker compose up -d` দিয়ে পুরো পরিবেশ চালু হয়	`docker compose ps` -এ সব সার্ভিস Up (healthy)
২	ব্রাউজারে frontend আসে, `/api/health` ok দেয়	`curl http://localhost:8080/health` → 200
৩	দুটি image-ই multi-stage, 200 MB-এর কাছাকাছি বা নিচে	`docker images` -এর আকার
৪	কোনো container root-এ চলে না	`docker exec <name> whoami` ≠ root
৫	MongoDB শুধু backend network-এ, বাইরে থেকে পৌঁছানো যায় না	হোস্ট থেকে `curl mongo:27017` ব্যর্থ; `docker network inspect`
৬	`down/up` -এর পর ডেটা থাকে	countDocuments() আগের মতোই
৭	Secret image-এ বা Git-এ নেই	`docker history` -এ secret নেই; `.gitignore` -এ `.env`
৮	Backup থেকে restore-এ ডেটা ফিরে আসে	Restore-এর পর count মেলে
৯	ভাঙা আপডেটে rollback কাজ করে	Rollback-এর পর `/health` 200
১০	Runbook সম্পূর্ণ ও অনুসরণযোগ্য	অন্য কেউ runbook পড়ে deploy/rollback করতে পারে

Grading Rubric (মূল্যায়ন)

ক্ষেত্র	ওজন	উত্তীর্ণ (Pass) মানে
Image ও multi-stage build	১৫%	দুটি image-ই multi-stage, চলে, আকার নথিভুক্ত
Compose ও network বিচ্ছিন্নতা	১৫%	চার সার্ভিস, দুটি network, mongo internal
Persistence, backup ও recovery	১৫%	Volume-এ ডেটা থাকে; backup/restore পরীক্ষা করা হয়েছে
নিরাপত্তা (non-root, secret, limit)	২০%	Non-root, secret image-এ নেই, resource limit আছে
Health check ও dependency	১০%	সব সার্ভিস healthy, ক্রম সঠিক
Registry, deploy, update/rollback	১৫%	Private registry-তে push, deploy script, rollback প্রমাণিত
Runbook ও documentation	১০%	Deploy, rollback, backup, incident — সব ধাপ লেখা

পাস মার্ক: মোট ৭০% ও নিরাপত্তা ক্ষেত্রে (non-root, secret, limit) অন্তত ৭০% — নিরাপত্তা ছাড়া production অসম্পূর্ণ।

ডেলিভারেবল

১. `compose.yaml` + দুটি Dockerfile + nginx config + `.env.example` (কাল্পনিক মানসহ)। ২. Deploy ও backup script। ৩. যাচাইয়ের আউটপুট (docker compose ps, curl, docker images, whoami, backup/restore log)। ৪.

Production runbook (deploy, rollback, backup, restore, incident)। ৫. সংক্ষিপ্ত প্রতিবেদন: কী বানালেন, কী শিখলেন, কী ঝুঁকি রয়ে গেল।

ক্যাপস্টোন — সমাধান নির্দেশিকা (Solution Guide)

ক্যাপস্টোন সমাধান নির্দেশিকা — প্রথমে নিজে চেষ্টা করুন, তারপর পড়ুন

> **সতর্কতা:** এই অংশটি শুধু আটকে গেলে বা নিজের সমাধান মিলিয়ে দেখার জন্য। প্রথমে ক্যাপস্টোনটি নিজে সম্পূর্ণ করার চেষ্টা করুন — সমাধান কপি করলে শেখা হবে না। নিচের কোডে সব মান কাল্পনিক (technova.example, CHANGE-ME-...)।

সমাধানের রূপরেখা

ধাপ ১ — Image: Backend-এর জন্য প্রজেক্ট ১-এর Dockerfile ব্যবহার করুন; frontend-এর জন্য `FROM node:20 AS build → npm ci → npm run build → FROM nginx:1.27-alpine → COPY --from=build /app/dist /usr/share/nginx/html`। দুটি image-ই build করে `docker images`-এ আকার লিখুন — backend ~120–180 MB, frontend ~50 MB (nginx alpine) হওয়ার কথা। **ধাপ ২ — Compose:** প্রজেক্ট ২-এর compose.yaml-কে ভিত্তি করে client সার্ভিস যোগ করুন (build: ./client, অথবা nginx-এ static পরিবেশন — দুটোর যেকোনো একটি, তবে পছন্দের কারণ লিখুন)। **ধাপ ৩ — Network:** `frontend-net` ও `backend-net` দুটি network; mongo শুধু backend-net-এ — `docker network inspect` দিয়ে যাচাই করুন। **ধাপ ৪ — Secret:** `.env`-এ `MONGO_PASSWORD=CHANGE-ME-MONGO-PASSWORD`, Compose-এ `${MONGO_PASSWORD}`; `docker history technova-api:1.0.0 --no-trunc | grep -i password`-এ কিছু পাওয়া যাবে না — এটাই প্রমাণ। **ধাপ ৫ — Deploy:** `docker compose pull && docker compose up -d && curl -fsS http://localhost:8080/health` — health check ব্যর্থ হলে deploy script exit 1 দিয়ে থামবে (প্রজেক্ট ৩-এর deploy.sh দেখুন)। **ধাপ ৬ — Backup:** `docker compose exec -T mongo mongodump -u admin -p CHANGE-ME-MONGO-PASSWORD --out /tmp/backup-2026-11-09`, তারপর নতুন volume-এ restore করে count মিলিয়ে দেখুন। **ধাপ ৭ — Rollback:** ভাঙা tag deploy করে `docker compose ps`-এ Exited দেখুন, তারপর আগের tag-এ `docker compose up -d` করে `curl /health` 200 দেখুন। **ধাপ ৮ — Runbook:** প্রতিটি পদ্ধতির ধাপ, কমান্ড, প্রত্যাশিত ফল ও "কাজ না করলে কী করবেন" — চারটি পাতায় লিখুন; ল্যাব ২৪-এর incident report-কে runbook-এর incident অংশে যোগ করুন।

সাধারণ সমাধান-ভুল

Frontend-এর build stage-এ `npm run build` ব্যর্থ: Vite-এর dependency devDependency-তে থাকে — build stage-এ `npm ci` (dev-সহ) দিন, runtime stage-এ নয়। **MongoDB health check ব্যর্থ:** `start_period: 20s` দিন; প্রথম চালুতে MongoDB 10–20 সেকেন্ড নেয়। **Nginx-এ `host not found in upstream api`:** `depends_on: api: condition: service_healthy` দিন — শুধু `depends_on: [api]` যথেষ্ট নয়। **Non-root-এ volume permission denied:** named volume ব্যবহার করুন, অথবা হোস্ট ফোল্ডারের মালিকানা UID 1000-এ মেলান।

নিজের সমাধান মিলিয়ে দেখার চেকলিস্ট

☐ দুটি image multi-stage ও 200 MB-এর কাছাকাছি ☐ সব সার্ভিস (healthy) ☐ mongo শুধু backend-net-এ ☐ ডেটা down/up-এর পর থাকে ☐ secret image-এ নেই ☐ backup/restore পরীক্ষা করা হয়েছে ☐ rollback পরীক্ষা করা হয়েছে ☐ runbook লেখা হয়েছে। আটটির মধ্যে অন্তত সাতটি হলে ক্যাপস্টোন উত্তীর্ণ — বাকি একটি কীভাবে ঠিক করবেন, তা প্রতিবেদনে লিখুন।

শেষ কথা

ওয়ার্কবুক শেষ — এরপর কী?

২৪টি ল্যাব, ১৫টি অ্যাসাইনমেন্ট, ৩টি মেজর প্রজেক্ট ও ক্যাপস্টোন শেষ করার পর আপনি শুধু Docker কমান্ড জানেন না — আপনি একটি অ্যাপকে image-এ বাঁধতে, network-এ যুক্ত করতে, volume-এ ডেটা রাখতে, Compose-এ পুরো পরিবেশ চালাতে, নিরাপদ করতে, backup নিতে, আপডেট ও rollback করতে, আর incident সামলাতে পারেন। এটাই container engineering-এর দৈনন্দিন কাজ। এরপর Kubernetes, CI/CD pipeline (Volume 05) ও production Linux (Volume 04)-এর সঙ্গে এই দক্ষতা জুড়ে দেখুন — container সেখানেই সবচেয়ে বেশি কাজে লাগে।

> **মনে রাখবেন:** Container অস্থায়ী, volume স্থায়ী; image অপরিবর্তনীয়, configuration পরিবেশভেদে বদলায়; secret কখনো image-এ যায় না; আর যেকোনো production পরিবর্তনের আগে backup ও rollback পরিকল্পনা থাকতে হয়। এই চারটি নিয়ম মনে রাখলে বেশিরভাগ container দুর্ঘটনা এড়ানো যায়।

ল্যাব VM-টি পরিষ্কার করুন: `docker compose down` (volume ছাড়া), অপ্রয়োজনীয় image `docker image prune` দিয়ে মুছুন, আর `docker volume ls` দিয়ে দেখুন কোন volume-এ এখনো দরকারি ডেটা আছে — সেটি মুছবেন না যতক্ষণ না backup নেওয়া আছে। শুভকামনা — TechNova-র পরের deployment আপনার হাতে।